

Implementation and Evaluation of Multi-Dimensional Defragmentation and Network-Aware Filtering Policies for Kubernetes Descheduler

Naufal Ichsan, Muhammad Raihan Akbar, Matthew Hotmaraja Johan Turnip, and Made Harta Dwijaksara

Faculty of Computer Science, Universitas Indonesia, Depok 16424, Indonesia
naufal.ichsan@ui.ac.id

ABSTRACT

Dynamic Kubernetes clusters accumulate multi-dimensional resource fragmentation that strands capacity, while compute-focused consolidation may disrupt busy workloads or move dependent microservices across degraded network links. This study presents a Kubernetes Descheduler architecture that combines resource defragmentation with actual-usage and network-aware eviction safeguards. The proposed *ResourceDefragmentation* policy evaluates CPU-memory shape using the Resource Imbalance Index (RII), Free-Space Index (FSI), and bin scores, then selects evictions based on feasible predicted scheduler targets. Actual-usage and network-aware filters respectively protect busy Pods and block movements toward latency-degrading destinations. Across five request-based resource scenarios, the policy reclaimed four Nodes in the underutilized scenario and three Nodes in the fragmented scenario. In the heterogeneous-drain scenario, it reduced Mean Absolute RII from the low-20% range to around 2% using two evictions. In the controlled single-replica workload, the actual-usage filter protected the busy API Pod by redirecting eviction to an idle candidate, preventing the failures observed when the API Pod was evicted. In the network-aware evaluation, the baseline ResourceDefragmentation behavior balanced compute resources but evicted sensitive workloads, causing latency spikes of up to ~ 167 ms. In contrast, the latency-aware filter intercepted these network-degrading actions, maintained baseline latency at ~ 8 ms, and still reduced Mean Absolute RII from the 50% range to around 17% by steering eviction toward non-critical Pods. Overall, the proposed architecture improves resource utilization while protecting active and network-sensitive workloads.

Keywords: Kubernetes, Descheduler, Resource Defragmentation, Runtime Resource Usage, Network Latency, Microservices

1. INTRODUCTION

This section introduces the background, research problems, objectives, contributions, and research position for the proposed Kubernetes Descheduler extensions.

1.1 Research Background

Kubernetes schedules Pods by filtering feasible Nodes and scoring the remaining alternatives. This process is effective for initial placement, but running Pods normally remain bound to their Nodes until they terminate or are recreated. As workloads scale and change, free CPU and memory can become scattered into incompatible shapes. Aggregate capacity may be sufficient while no individual Node can accommodate a new Pod, a condition commonly described as resource fragmentation.

Fragmentation affects schedulability, utilization, and infrastructure cost. A Node may retain CPU without enough memory, while another retains memory without enough CPU. These asymmetric residuals strand capacity because the two dimensions cannot be allocated independently (Xiao et al., 2013; Guo et al., 2025). Directed Pod eviction can repair such placement states and reduce the number of Nodes required by the same workload (Larsson et al., 2023).

The Kubernetes Descheduler provides a post-placement correction mechanism by evicting selected Pods and allowing the kube-scheduler to place their replacements (Kubernetes SIG Scheduling, nd). For node reclamation, the important question is not merely whether a source Node is lightly utilized, but whether an evicted Pod has a feasible target that produces denser and better-shaped packing. This requires source-node assessment, target prediction, and an eviction budget that limits disruption.

However, eviction is not free. To

remain mathematically aligned with the *kube-scheduler*'s system, a defragmentation policy must evaluate Pods primarily through their static declared resource requests rather than actual usage, because these static requests dictate placement feasibility and bin-packing shape. In practice, a Pod selected as highly beneficial for resource defragmentation based on these static requests may still be serving high live demand at the moment eviction is attempted. Evicting such a Pod introduces a transient service cost: the original Pod is terminated, the replacement must wait for scheduling, image preparation, startup, readiness, and traffic reattachment, and during this interval the application may experience reduced capacity or degraded performance. Therefore, a practical defragmentation architecture also requires a pre-eviction filter that checks actual resource usage before allowing an eviction to proceed.

Resource packing can also affect application communication. Moving dependent microservices to distant or degraded Nodes may improve compute packing while increasing latency. Communication-aware placement studies show that network topology, observed latency, and service dependencies can materially affect placement quality (Marchese and Tomarchio, 2022b,a; Chen et al., 2024). A practical node reclamation architecture therefore also needs a network-aware guardrail.

This research develops a defragmentation-focused Descheduler architecture with complementary guardrails. *ResourceDefragmentation* identifies drain candidates using utilization and CPU-memory shape, predicts feasible scheduler targets, and chooses a Pod using the projected bin score of its predicted landing Node. An actual-usage pre-eviction filter blocks evictions when the selected Pod is currently

consuming high live resources, reducing the risk of performance degradation during replacement. A network-aware filtering layer further blocks evictions whose feasible destinations would degrade communication locality.

1.2 Problem Statement

The study addresses the following problems, bounded specifically within the post-placement correction capabilities of the Kubernetes Descheduler rather than initial placement logic:

1. **Resource fragmentation:** CPU and memory may remain available in aggregate but become scattered across Nodes in incompatible shapes. CPU-heavy and memory-heavy placements can strand one resource dimension behind another, while scheduler bin-packing scores only act during initial placement and do not repair fragmentation that has accumulated in an already running cluster.
2. **Actual-usage eviction risk:** A Pod that looks beneficial to evict from a resource-request and bin-packing perspective may still be handling high live demand when eviction is attempted. Evicting such a Pod can temporarily reduce application capacity because the replacement Pod must be scheduled, started, become ready, and reconnect to traffic before service capacity is restored. Therefore, defragmentation decisions need a pre-eviction guardrail that considers actual resource usage, not only declared resource requests.
3. **Network-locality risk:** Eviction decisions that focus only on cluster re-balance can separate network dependent services and increase communi-

cation overhead, especially in multi-node or multi-zone deployments.

1.2.1 Research Question

- How can a Descheduler policy identify and correct CPU-memory fragmentation through directed Pod eviction, and how effectively does it reclaim nodes and reduce stranded capacity compared to utilization-based baselines?
- How effectively can an actual-usage pre-eviction filter reduce the risk of disrupting high-demand Pods without severely compromising the Descheduler's ability to reduce cluster fragmentation?
- How effectively can a network-aware pre-eviction filter protect communication latency between dependent services, and what is the trade-off with overall cluster fragmentation reduction?

1.2.2 Research Limitation

To maintain focus and define the scope of this study, the research limitations are established as follows:

- **Infrastructure:** Evaluation is conducted on a small-scale Kubernetes cluster using AWS EC2 nodes, with additional logical topology separation to emulate multi-region network conditions.
- **Workload load generation:** Actual workload demand used to evaluate the pre-eviction actual-usage filter is generated using k6 load testing scenarios, rather than relying on naturally occurring production traffic.
- **Network:** Experiments assume standard AWS networking without specialized hardware acceleration such as SR-IOV.

- **Provisioning:** The cluster is provisioned using kubeadm to approximate production behavior, but excludes enterprise-grade storage features and complex service meshes unless specifically required for the evaluation scenario.
- **Framework dependencies:** The solution is implemented within the Kubernetes Descheduler framework; it determines which Pods should be evicted, while final placement remains the responsibility of the kube-scheduler.
- **Scheduler alignment:** Because the Descheduler operates in isolation from the primary scheduler, the ultimate success of an eviction depends on the kube-scheduler's configuration. This study utilizes standard default scheduler profiles such as *LeastAllocated* for spreading, or *MostAllocated* paired with *BalancedAllocation* for consolidating. Consequently, if the default scheduler is not perfectly aligned with the descheduler's intent, it may place evicted pods onto suboptimal nodes.
- **Scope of network awareness:** Network-aware logic is treated as an eviction scoring or filtering consideration, not as a complete traffic-engineering or routing solution.

1.3 Research Objectives

This research proposes a three-step descheduler workflow to resolve the interconnected problems of resource fragmentation, disruption risk, and network locality. The specific objectives are:

- **Resource Defragmentation Balancing Plugin:** Design a defragmentation-focused Descheduler policy that

combines utilization and resource-shape assessment (using RII, FSI, and bin scores) to identify fragmented nodes and predict optimal landing targets. This objective includes evaluating the policy's effectiveness in freeing up infrastructure, reducing stranded resources, and increasing usable capacity while ensuring cluster stability.

- **Actual Resource Utilization Pre-eviction filter:** Design and integrate a pre-eviction gate that prevents high-demand Pods from being evicted by verifying live traffic metrics. This objective includes evaluating the filter's effect on eviction safety and its trade-off with overall cluster fragmentation reduction.
- **Network Aware Pre-eviction filter:** Design and integrate a network-aware gate that protects communication locality by verifying that predicted destinations will not degrade latency. This objective includes evaluating the filter's impact on network performance and its trade-off with cluster fragmentation reduction.

1.4 Research Contributions

- Implemented and evaluated *ResourceDefragmentation*, a defragmentation-focused Descheduler policy that repairs post-placement scatter without replacing the kube-scheduler. The evaluation demonstrates its effectiveness in reducing fragmentation, improving usable capacity, and supporting cluster consolidation while preserving cluster stability.
- Implemented and evaluated actual-usage cost filtering to guard high-demand Pods from disruptive

eviction and assess its trade-off with fragmentation reduction.

- Implemented and evaluated network-aware cost filtering to guard network-dependent services from harmful movement and assess its trade-off with fragmentation reduction.

1.5 Research Position

Prior studies have addressed several parts of the problem targeted by this research. Early work on dynamic resource allocation, such as (Xiao et al., 2013), shows that multi-dimensional resource skewness can reduce effective server utilization. However, these approaches were primarily designed for traditional virtual-machine environments and often rely on static threshold-based heuristics. More recent work has studied container resource defragmentation and task scheduling using optimization or learning-based methods (Guo et al., 2025; Bagaa et al., 2024), but these studies are commonly oriented toward placement or scheduling decisions rather than long-term post-placement correction in Kubernetes clusters.

Kubernetes-specific research has also shown that resource fragmentation can emerge over time and that directed Pod eviction can improve utilization (Larsson et al., 2023). Nevertheless, fragmentation-aware consolidation by itself is not sufficient for production-oriented microservice workloads. A Pod that is attractive to evict from a resource-request perspective may still be serving high live demand when the eviction is executed. In that case, a consolidation decision that appears beneficial at the placement level may impose temporary service capacity loss during termination, rescheduling, startup, readiness recovery, and traffic reattachment. Network-aware placement studies show that service

communication patterns and network conditions can affect placement quality (Marchese and Tomarchio, 2022b,a; Chen et al., 2024), but these works do not fully address how such considerations should be integrated into a Descheduler-based consolidation policy.

Another related direction is group or gang-style movement, where dependent services are moved together to preserve affinity. While this approach can protect communication locality, it may increase the risk of resource deadlock or failed movement because the scheduler must find enough capacity for a larger combined group. For a fragmentation-oriented Descheduler policy, this can worsen the packing problem instead of solving it. Therefore, this research positions itself away from strictly atomic group migration and instead adopts a non-atomic, individual scoring paradigm.

This study addresses the gap by developing a Descheduler-based framework that combines multi-dimensional resource imbalance analysis, actual-usage eviction safety, and network-aware eviction considerations. The proposed direction uses a mathematical imbalance index (RII_i) and a resource defragmentation algorithm to target specific Pods that contribute to stranded capacity, while adding actual-usage cost filtering and network-aware filtering so that consolidation decisions are less likely to disrupt high-demand Pods or degrade communication latency.

The research position can be summarized as follows:

- It broadens resource balancing beyond simple utilization thresholds by considering multi-dimensional imbalance and stranded capacity.
- It treats actual runtime usage as a pre-eviction safety guardrail, separating scheduler feasibility from the

service cost of disrupting a busy Pod.

- It incorporates network-aware considerations as an individual, score-based pre-eviction guardrail rather than relying on atomic group migration. This connects fragmentation reduction with network-locality preservation while avoiding the placement deadlocks associated with moving entire service groups.

1.6 Research Steps

1. Review current literature on Kubernetes scheduling, resource fragmentation, node reclamation, descheduler policies, and network-aware placement.
2. Design and implement the three core components: the *ResourceDefragmentation* strategy, the Actual-Usage filter, and the Network-Aware filter within the Kubernetes Descheduler framework.
3. Evaluate the standalone effectiveness of the *ResourceDefragmentation* strategy against baseline utilization strategies across controlled scenarios.
4. Evaluate the individual safety trade-offs of the Actual-Usage and Network-Aware pre-eviction filters using runtime demand and latency-sensitive workloads.
5. Conduct a realistic end-to-end scenario to evaluate the combined workflow of all three components operating as a single unified pipeline.
6. Analyze the overall outcomes, including node reclamation, stranded capacity reduction, service disruption avoidance, and network-latency preservation.

2. LITERATURE REVIEW

This section reviews the theoretical concepts and prior work that form the foundation of this study, including Kubernetes resource management, resource fragmentation, Descheduler mechanisms, predicted-target eviction selection, runtime load-aware eviction guarding, and communication-aware placement.

2.1 Kubernetes Architecture and Resource Management

Kubernetes distributes Pods across worker Nodes. For CPU and memory, a container may declare a resource request, a resource limit, both, or neither. Requests represent the resource baseline used by the scheduler when determining whether a Pod fits on a Node, whereas limits are runtime enforcement ceilings applied by the kubelet and container runtime (Kubernetes Authors, nde). Kubernetes uses effective Pod requests when evaluating placement feasibility and resource allocation on each Node. Resource limits do not participate in this fit calculation; they define runtime enforcement ceilings after the Pod is running. Scheduler scoring strategies such as *MostAllocated* and *RequestedToCapacityRatio* may then use request-based allocation to favor denser packing (Kubernetes Authors, ndd).

Requests and limits also contribute to Pod Quality of Service (QoS) classification. A Pod is *Guaranteed* when every container declares CPU and memory requests equal to its corresponding limits, while Pods with partial or unequal declarations are generally *Burstable*. A Pod whose containers declare neither CPU nor memory requests or limits is *BestEffort* (Kubernetes Authors, ndc). If a limit is declared without a request, Kubernetes can use that limit as the effective request for the resource. A namespace *LimitRange* may also assign de-

fault requests or limits during Pod admission. These defaults become part of the Pod’s effective resource configuration (Kubernetes Authors, nda).

These decisions are made when a Pod is scheduled. A running Pod is not continuously relocated when the cluster state changes, so an initially acceptable layout can become inefficient after scale-outs, terminations, and replacement events. This separation between initial placement and later cluster state motivates post-placement correction.

2.2 Resource Fragmentation and Stranded Capacity

Resource fragmentation occurs when free capacity is divided among Nodes in shapes that do not match incoming workloads. A cluster can have enough aggregate CPU and memory while no individual Node satisfies both dimensions of a Pod request. This makes CPU and memory placement a multi-dimensional problem rather than a scalar load-balancing problem.

CPU-memory skew is one source of stranded capacity. If one Node is CPU-heavy and another is memory-heavy, each leaves a different residual resource that cannot be consumed alone. The absolute difference between normalized CPU and memory fractions provides a direct shape indicator, while the product of free CPU and memory fractions represents the jointly usable free-space region. Prior work shows that directed eviction can repair fragmented Kubernetes layouts and support Node reclamation (Larsson et al., 2023).

2.3 Resource Defragmentation and Multi-Resource Bin Packing

Resource packing seeks to place workloads on fewer active Nodes so that empty Nodes can be removed or powered down.

In a multi-resource setting, dense packing alone is insufficient: a target can be highly utilized but badly skewed. A useful packing score should therefore represent both density and CPU-memory balance.

The policy in this research uses normalized CPU and memory fractions. Average utilization represents density, while a bin score combines density with the absolute distance between the two dimensions. The Resource Imbalance Index (RII) preserves the direction of skew, and the Free-Space Index (FSI) measures the rectangular CPU-memory headroom. Together these quantities identify Nodes that are lightly loaded, poorly shaped, or under pressure from a narrow free-space region.

2.4 Kubernetes Descheduler

The Kubernetes Descheduler evicts Pods that violate a policy or contribute to an undesirable placement state, after which normal controllers recreate the Pods and kube-scheduler places them again (Kubernetes SIG Scheduling, nd). It provides an appropriate extension point for node reclamation because it can repair placement without modifying the scheduler.

An eviction policy must nevertheless account for the scheduler boundary. Deleting a Pod does not guarantee that it moves to a useful Node; it may become Pending, return to its origin, or land on a target that worsens fragmentation. A node reclamation-oriented policy therefore benefits from predicting feasible non-origin targets and rejecting movements that exceed a configured ceiling or pack downward.

Upstream utilization policies provide important baselines. *HighNodeUtilization* packs by draining low-utilization Nodes toward more utilized Nodes, but it does not explicitly optimize CPU-memory stranding. This distinction is central to the

evaluation in Section 4.

2.5 Predicted-Target Eviction-Candidate Selection

Eviction-candidate selection can be formulated around the expected landing state. For every evictable Pod on a drain candidate, *predictSchedulerTarget* simulates request-based placement on non-origin Nodes that satisfy scheduler constraints. Feasible targets must have sufficient CPU and memory, remain below a node reclamation ceiling after placement, and be at least as packed as the source.

The target with the best projected CPU-memory bin score is retained for each Pod. The Pod whose predicted target has the highest score is selected for eviction. This single criterion aligns the action directly with the desired destination state: the selected movement should place complementary resource shapes together while preserving node reclamation pressure.

2.6 Runtime Load-Aware Eviction Guard

Request-based resource information is sufficient for scheduler feasibility and bin-packing analysis because Kubernetes places Pods according to their effective requests rather than their instantaneous consumption. However, the operational cost of eviction depends on the Pod’s runtime state. A Pod that is attractive from a defragmentation perspective may still be handling high live demand when the Descheduler attempts to evict it. Load-aware orchestration addresses this limitation by incorporating runtime resource usage into placement decisions. For example, (Marchese and Tomarchio, 2024) propose a Kubernetes orchestration strategy that continuously considers dynamic resource usage patterns when placing microservice applications in shared clusters.

This research applies the same prin-

ciple at the eviction boundary rather than replacing the scheduler. The defragmentation policy evaluates source Nodes, target feasibility, and projected CPU-memory shape in request space because those quantities determine whether the replacement Pod can be scheduled. The actual-usage guard then compares live consumption with the same request baseline to determine whether the selected Pod is currently active relative to the capacity used for placement. Resource limits are not used as denominators because usage relative to a limit indicates proximity to CPU throttling or a memory out-of-memory boundary. This represents resource saturation rather than the busy-Pod protection objective of this plugin. Limits can nevertheless affect observed usage indirectly by throttling CPU or terminating a container that exceeds its memory limit (Kubernetes Authors, nde).

This Descheduler guard also has a different objective from kubelet node-pressure eviction. Under node pressure, the kubelet ranks Pods to free a pressured resource based on whether usage exceeds requests, Pod Priority, and usage relative to requests. QoS influences this ranking through the requests and priority characteristics of each Pod (Kubernetes Authors, ndb). The Actual Usage Evictor instead decides whether a strategy-selected eviction should proceed without disrupting a currently busy Pod. This separation preserves request-space defragmentation while adding a runtime safety decision immediately before eviction.

2.7 Communication-Aware and Network-Aware Placement

Microservices exchange traffic across service dependencies. Placement across distant zones or degraded links can increase latency even when compute resources fit. Communication-aware Kubernetes

studies therefore incorporate topology, traffic, or measured network conditions into placement decisions (Marchese and Tomarchio, 2022b,a; Chen et al., 2024; Pusztai et al., 2021).

This research applies network awareness as an eviction guardrail. A global pre-eviction filter compares the current communication cost with feasible alternatives, while strategy-level filtering can evaluate a narrower target set. The design keeps resource improvement and network protection separable but composable.

2.8 Current Approaches and Research Gap

Existing mechanisms address individual parts of the problem. Scheduler scoring improves new placement, utilization-based descheduling drains Nodes according to thresholds, fragmentation research motivates directed eviction, load-aware orchestration highlights the importance of runtime demand, and network-aware schedulers protect communication locality. The remaining gap is a practical Descheduler pipeline that combines:

- source assessment using utilization and CPU-memory shape;
- RII/FSI-based drain ordering;
- request-feasible scheduler-target prediction;
- a direct projected-target balance criterion;
- bounded, convergent eviction;
- actual-usage pre-eviction filtering for busy Pods; and
- network-aware filtering for latency-sensitive workloads.

ResourceDefragmentation and the Network-Aware Evictor are designed to fill this gap while leaving final binding to the standard kube-scheduler.

3. RESEARCH METHODOLOGY

This section presents the design and implementation of the Descheduler components developed in this research: the defragmentation-focused *ResourceDefragmentation* policy, the Actual Usage Evictor runtime pre-eviction filter, and the Network-Aware Evictor with topology-based and latency-based cost models.

Several components are implemented as descheduler *EvictorPlugin* extensions. In this framework, the *Filter* stage performs candidate filtering, while the *PreEvictionFilter* stage is executed immediately before an eviction request is submitted. Pre-eviction filters can be chained, therefore a candidate Pod must pass all enabled filters before the descheduler performs the eviction. This mechanism is used by both the Actual Usage Evictor and the Network-Aware Evictor as safety gates around strategy-driven eviction decisions.

3.1 Proposed Descheduler Extension Architecture

This section summarizes the proposed descheduler schemes and explains how they relate to the Kubernetes Descheduler eviction workflow.

This research proposes three descheduler extensions that operate at different points of the Kubernetes Descheduler eviction decision flow. The extensions are designed as independent components rather than as a tightly coupled pipeline. Each component can be enabled separately according to the cluster operator’s objective, and multiple components can be combined within the same

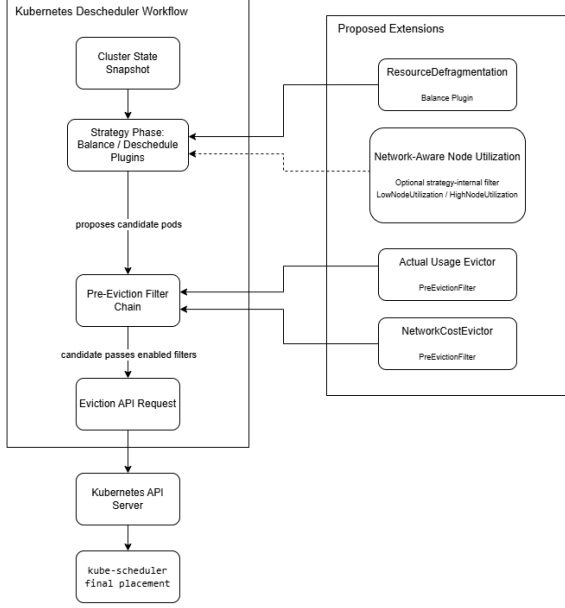


Figure 1. Proposed independent descheduler extensions and their extension points within the Kubernetes Descheduler eviction workflow.

descheduler profile when resource defragmentation, runtime-safety protection, and network-locality protection are required together.

Figure 1 summarizes the position of each proposed component in the descheduler architecture. The *ResourceDefragmentation* component is implemented as a *Balance* plugin. It participates in the strategy phase by analyzing the requested CPU-memory layout of the cluster, identifying fragmented or underutilized source nodes, predicting feasible landing destinations, and selecting eviction candidates that are expected to improve resource packing.

The *Actual Usage Evictor* is implemented as a *PreEvictionFilter*. It does not select source nodes, destination nodes, or eviction candidates. Instead, it evaluates a candidate Pod immediately before eviction by checking live CPU and memory usage. This allows the descheduler to block eviction of Pods that are currently active, even when the candidate was selected by

either the proposed *ResourceDefragmentation* strategy or by another descheduler strategy.

The *Network-Aware Evictor* provides communication-cost protection for eviction decisions. At the global layer, the *NetworkCostEvictor* acts as a *PreEvictionFilter* that evaluates whether evicting a candidate Pod may lead to a placement with worse communication cost. At the strategy layer, network-aware evaluation is integrated into node-utilization strategies, namely *LowNodeUtilization* and *HighNodeUtilization*, so that network cost is considered earlier when candidate destinations are derived. This component can therefore be used independently with existing descheduler strategies, combined with node-utilization strategies at the strategy layer, or composed with the proposed *ResourceDefragmentation* policy through the global pre-eviction filter layer.

When all proposed components are enabled together, the descheduler forms a composable eviction decision flow: a strategy proposes a candidate Pod, runtime usage filtering verifies whether the Pod is safe to evict, network-aware filtering verifies whether the eviction preserves communication locality, and only then is the eviction request submitted to the Kubernetes API. However, this combined flow is only one deployment configuration; the proposed components remain independently usable extensions.

3.2 ResourceDefragmentation Policy Design

ResourceDefragmentation is a *Balance* plugin for post-placement consolidation. It builds a snapshot of worker Nodes based on Pod resource requests, finds source Nodes that are either sparse or poorly shaped in CPU-memory space, predicts feasible landing Nodes for their Pods, and evicts the Pod whose predicted scheduler target

has the strongest projected CPU-memory bin score. The kube-scheduler remains responsible for final binding.

The experimental configuration explicitly uses declared Pod requests. This keeps Node assessment, feasibility checks, and target prediction aligned with the admission model used by kube-scheduler.

3.2.1 System Overview

Figure 2 shows the end-to-end control flow of the *ResourceDefragmentation* plugin, from the entry point of the Descheduler *Balance* function to the eviction decision. The plugin operates in two phases: an assessment-and-ordering phase that builds the drain-candidate set, and a bounded eviction loop that relocates Pods from those candidates.

In the first phase, the plugin scans every worker Node. Control-plane Nodes are skipped, and for each remaining Node it computes the average utilization and bin score. A Node whose average utilization and bin score both stay at or above the drain threshold is well-packed and is skipped; otherwise the Node is scored with its Resource Imbalance Index (RII), Free-Space Index (FSI), and priority P_n , added to the drain-candidate set, and the set is sorted in descending priority.

In the second phase, the plugin iterates over the ordered candidates. The loop stops once the per-sweep eviction budget is reached. A Node already marked as a bin, that is, a recent eviction target, is skipped so the same sweep does not drain a Node it has just packed. Otherwise the plugin selects the Pod whose predicted scheduler target has the highest projected bin score; if a feasible target exists the Pod is evicted and the local cache and bin set are updated, and if not the Node is skipped. The remaining subsections formalize each stage: Section 3.2.2 defines the Node-state metrics,

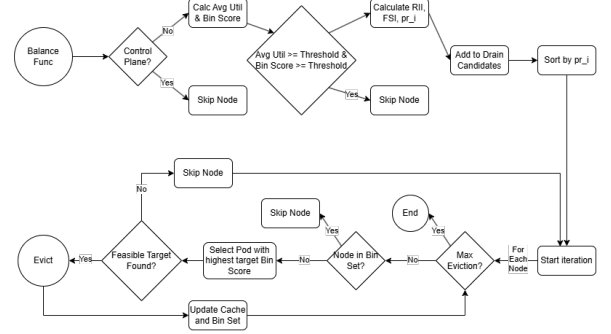


Figure 2. High-level control flow of the *ResourceDefragmentation* plugin, from Node assessment and drain-candidate ordering to the bounded predicted-target eviction loop.

Section 3.2.3 the drain-candidate detection and ordering, Section 3.2.4 the predicted scheduler target, and Section 3.2.5 the eviction-candidate selection.

3.2.2 Node-State Metrics

For Node n , let $u_{n,c}$ and $u_{n,m}$ denote normalized requested CPU and memory:

$$u_{n,c} = \frac{R_{n,c}}{C_{n,c}}, \quad u_{n,m} = \frac{R_{n,m}}{C_{n,m}},$$

where $u_{n,c}$ is the normalized CPU utilization of Node n , $u_{n,m}$ is the normalized memory utilization of Node n , $R_{n,c}$ is the sum of CPU requests from all Pods on Node n , $R_{n,m}$ is the sum of memory requests from all Pods on Node n , $C_{n,c}$ is the allocatable CPU capacity of Node n , and $C_{n,m}$ is the allocatable memory capacity of Node n . Average utilization represents density:

$$U_n^{avg} = \frac{u_{n,c} + u_{n,m}}{2},$$

where U_n^{avg} is the average utilization of Node n , representing overall resource density by taking the mean of normalized CPU and memory utilization.

The bin score is a custom composite scoring function derived from the

standard multi-dimensional bin-packing heuristics used in the Kubernetes scheduling plugins (Kubernetes Authors, n.d.). It fuses two signals that those plugins apply separately, namely the density preference of the MostAllocated strategy, which favors well-filled Nodes, and the per-dimension balance preference of the BalancedAllocation strategy, which penalizes Nodes whose CPU and memory utilization diverge. Combining density and dimensional balance into a single quantity yields

$$B_n = \frac{\frac{u_{n,c}+u_{n,m}}{2} (1 - |u_{n,c} - u_{n,m}|)}{2},$$

where B_n is the bin score of Node n . The first factor $\frac{u_{n,c}+u_{n,m}}{2}$ is the density term inherited from MostAllocated, while the penalty factor $1 - |u_{n,c} - u_{n,m}|$ is the balance term inherited from BalancedAllocation, which reduces the score when CPU and memory utilization diverge. A dense and balanced Node therefore has a high B_n , whereas an empty or strongly skewed Node has a lower score. The signed Resource Imbalance Index, adapted to normalized request fractions from the server imbalance index of (Guo et al., 2025), identifies the skew direction:

$$RII_n = u_{n,c} - u_{n,m},$$

where RII_n is the Resource Imbalance Index of Node n . A positive value indicates CPU-heavy skew (CPU utilization exceeds memory utilization), while a negative value indicates memory-heavy skew. The Free-Space Index, adapted from the free-space size index of (Guo et al., 2025) by expressing the headroom in normalized rather than absolute units, represents joint CPU-memory headroom:

$$FSI_n = (1 - u_{n,c})(1 - u_{n,m}),$$

where FSI_n is the Free-Space Index of Node n , representing the product of remaining CPU capacity $(1 - u_{n,c})$ and remaining memory capacity $(1 - u_{n,m})$. A higher FSI_n indicates a larger jointly usable headroom region, while a low FSI_n suggests constrained free space in at least one dimension.

3.2.3 Drain-Candidate Detection and Ordering

A worker Node with at least one evictable Pod becomes a drain candidate when either its average utilization or its bin score is below the configured drain threshold τ :

$$U_n^{avg} < \tau \quad \vee \quad B_n < \tau.$$

The disjunction allows the policy to act on two resource-only states. A lightly utilized but balanced Node can be drained to free infrastructure, while a more utilized but badly skewed Node can be drained because its remaining headroom is hard to reuse.

Drain candidates are processed in descending priority:

$$P_n = 0.5 |RII_n| + 0.5 \frac{1}{FSI_n + 1e-10}.$$

The first term emphasizes dimensional skew. The second term emphasizes a narrow jointly usable free-space region. Following the priority formulation of (Guo et al., 2025), the constant is fixed at $1e-10$ and is added to the free-space term of *every* Node as a numerical guard against division by zero, rather than being applied conditionally only when $FSI_n = 0$. Because $1e-10$ is negligible relative to any non-zero $FSI_n \in (0, 1]$, it leaves the relative ordering of drain candidates unchanged for partially loaded Nodes, while bounding the term to a large finite value of $1e+10$ when a Node has no

jointly usable headroom ($FSI_n = 0$), so that fully constrained Nodes receive the highest priority. Utilization and bin score decide whether a Node enters the drain set, while RII and FSI decide which drain candidate is handled first.

The two weights of 0.5 generalize the priority formulation of (Guo et al., 2025), which writes the priority as $w_p |RII_n| + (1 - w_p) / (FSI_n + \epsilon)$ for a tunable weight w_p . Setting $w_p = 0.5$ treats dimensional skew and free-space scarcity as co-equal fragmentation signals, so that neither criterion is privileged a priori in the absence of a workload-specific reason to favor one. The two terms nonetheless occupy different numeric ranges: $|RII_n| \in [0, 1]$, whereas $1/(FSI_n + 1e-10) \in [1, 1e+10]$. The free-space term therefore dominates the ordering for densely packed Nodes that have little joint headroom, while $|RII_n|$ acts as a secondary, skew-based tie-breaker among Nodes with comparable free space.

Adjusting this weighting does not change the descheduler as a whole. The priority P_n governs only the *order* in which already-selected drain candidates are processed; it does not decide which Nodes enter the drain set, whether a predicted target is feasible, the pack-upward guard, the utilization ceiling, or the bounded convergence loop. Re-weighting therefore alters the drain trajectory, namely which Node is handled first and possibly the number of passes required, and can shift the final layout only at the margin, for instance when the pass budget is exhausted before convergence. Because every pass re-evaluates the cluster state and the loop terminates only when a pass produces zero evictions, the policy’s feasibility, safety, and convergence properties are independent of w_p , and the converged layout remains stable across reasonable weightings. The weight is thus a tuning knob for ordering and effi-

ciency rather than a parameter that redefines what the descheduler is permitted to evict.

3.2.4 Predicted Scheduler Target

For every evictable Pod p on source Node s , the plugin examines each non-origin worker Node as a potential relocation target. Let $r_{p,c}$ and $r_{p,m}$ denote the CPU and memory requests of p . Placing p on a target Node t yields the projected utilization

$$u'_{t,c} = \frac{R_{t,c} + r_{p,c}}{C_{t,c}}, \quad u'_{t,m} = \frac{R_{t,m} + r_{p,m}}{C_{t,m}}.$$

A non-origin Node t is admissible only if it is schedulable and p tolerates its taints. An admissible Node is a feasible target when it additionally satisfies the request-fit, utilization-ceiling, and pack-upward conditions, respectively:

$$\begin{aligned} R_{t,c} + r_{p,c} &\leq C_{t,c}, & R_{t,m} + r_{p,m} &\leq C_{t,m}, \\ u'_{t,c} &\leq \theta, & u'_{t,m} &\leq \theta, \\ \min(u'_{t,c}, u'_{t,m}) &\geq \min(u_{s,c}, u_{s,m}). \end{aligned}$$

where θ is the configured target-utilization ceiling, and $u_{s,c}$ and $u_{s,m}$ are the current CPU and memory utilization of source Node s . The request-fit condition ensures that the projected requests do not exceed allocatable capacity in either dimension. The utilization-ceiling condition caps the projected utilization so that consolidation does not overload a target. The pack-upward guard requires the least-utilized dimension of the target to be at least as loaded as the least-utilized dimension of the source, which prevents an eviction from merely relocating a Pod to an emptier Node.

Let $\mathcal{F}(p)$ denote the set of all feasible targets for p . The function *predictSchedulerTarget* returns the feasible Node that maximizes the projected bin score after receiving p :

Table 1. ResourceDefragmentation Plugin Arguments

Parameter	Type	Experiment	Purpose
<i>maxEvictions</i>	integer	50	Upper bound on successful eviction attempts in one pass.
<i>consolidationThreshold</i>	float	0.40	Marks a Node as a drain candidate when average utilization or bin score is below this value.
<i>consolidationTarget</i>	float	0.90	Maximum projected CPU or memory utilization on a target Node.
<i>usageMode</i>	string	<i>requests</i>	Pins the experiment to declared requests so all compared policies use the same signal.
<i>namespaces</i>	object	exclude system	Restricts candidate Pods and excludes infrastructure namespaces.

$$t_p = \arg \max_{t \in \mathcal{F}(p)} B'_t,$$

where B'_t is the bin score B_t evaluated under the projected utilization $(u'_{t,c}, u'_{t,m})$. If $\mathcal{F}(p) = \emptyset$, Pod p has no feasible target and is not evicted.

3.2.5 Predicted-Target Eviction-Candidate Selection

For candidate Pod p , let t_p be its predicted target and B'_{t_p} the target bin score after placement. The selected eviction candidate is:

$$p^* = \arg \max_p B'_{t_p}.$$

This is the only eviction-candidate selection criterion after target feasibility is checked. It directly prefers a movement that produces the best projected CPU-memory shape at the expected destination. Candidates without a feasible target are excluded. After a successful eviction, the local source and predicted-target snapshots are updated before the next decision. Predicted targets are marked as bins so the same pass does not subsequently drain a Node that has just received packed workload.

Appendix A summarizes the complete execution flow.

3.2.6 Plugin Configuration

```
1 apiVersion: descheduler/v1alpha2
2 kind: DeschedulerPolicy
3 profiles:
```

```
4   - name: default
5     pluginConfig:
6       - name: ResourceDefragmentationC2
7         args:
8           maxEvictions: 50
9           consolidationThreshold: 0.40
10          consolidationTarget: 0.90
11          usageMode: requests
12          namespaces:
13            exclude:
14              - kube-system
15        plugins:
16          balance:
17            enabled:
18              - ResourceDefragmentationC2
```

Listing 1. Request-based ResourceDefragmentation configuration using the ResourceDefragmentationC2 plugin.

3.3 Actual Usage Evictor Policy Design

The Actual Usage Evictor is designed to prevent eviction of Pods that are currently busy even when they are selected by a descheduler strategy. Resource-balancing strategies commonly reason about declared requests or node-level utilization, while the real workload state may change at runtime. Evicting a Pod during high CPU or memory activity can interrupt active processing and may increase service disruption. The Actual Usage Evictor addresses this issue by evaluating the current Pod metrics immediately before eviction.

This plugin does not choose source nodes, destination nodes, or eviction candidates. Instead, it acts as a global pre-eviction safety gate. A candidate Pod is allowed to continue only when its current CPU and memory usage are both below the configured thresholds relative to its declared requests.

3.3.1 System Design

The Actual Usage Evictor is implemented as an *EvictorPlugin*. Its *Filter* function is a no-op and always returns true, because runtime usage is evaluated only at the *PreEvictionFilter* stage. This design separates general candidate eligibility from

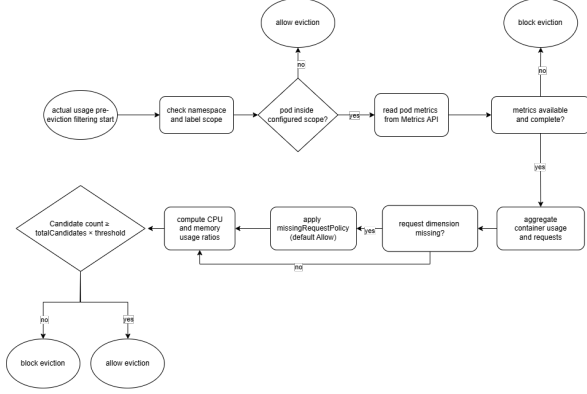


Figure 3. Actual Usage Evictor pre-eviction filtering flow.

runtime safety checking: the strategy may first select a Pod as an eviction candidate, but the Actual Usage Evictor still has the opportunity to block the eviction before the request is submitted.

The plugin can be scoped using namespace rules and a Kubernetes label selector. When a Pod is outside the configured scope, the plugin allows the eviction without reading metrics. When both namespace and label selector scopes are configured, both conditions must match before the Pod is evaluated. This allows the filter to protect only selected workloads, for example latency-sensitive applications, while leaving other workloads unaffected.

For Pods inside the configured scope, the plugin reads Pod metrics through the descheduler metrics collector and the Kubernetes Metrics API. The metrics of all containers in the Pod are matched against the Pod specification and aggregated before the usage ratios are calculated.

3.3.2 Runtime Usage Model

For a Pod p , let $R_{p,c}$ and $R_{p,m}$ denote its effective requested CPU and memory aggregated across all regular containers, and let $A_{p,c}$ and $A_{p,m}$ denote the corresponding actual usage obtained from Pod met-

rics. A Pod-level request dimension is complete only when every regular container declares an effective request for that resource. For complete CPU and memory dimensions, the runtime usage ratios are defined as:

$$\rho_{p,c} = \frac{A_{p,c}}{R_{p,c}}, \quad \rho_{p,m} = \frac{A_{p,m}}{R_{p,m}},$$

where $\rho_{p,c}$ is the actual CPU usage ratio and $\rho_{p,m}$ is the actual memory usage ratio. CPU is aggregated in millicores, while memory is aggregated in bytes. The Actual Usage Evictor reads effective requests from the Pod object stored by the API server. Therefore, requests introduced during admission, including defaults from a *LimitRange*, are included without requiring the plugin to query *LimitRange* objects directly.

The ratios intentionally compare usage with requests rather than with resource limits or Node capacity. Requests are the workload baseline used for scheduler placement and request-space defragmentation, so the ratio expresses whether a Pod is active relative to that baseline. Limits instead define runtime enforcement ceilings. Replacing a request denominator with a limit would change the decision from busy-Pod protection to detecting proximity to CPU throttling or a memory out-of-memory boundary.

Given CPU threshold θ_c and memory threshold θ_m , the eviction is blocked when either resource reaches or exceeds its threshold:

$$\rho_{p,c} \geq \theta_c \quad \vee \quad \rho_{p,m} \geq \theta_m.$$

Therefore, a Pod is allowed to be evicted only when both usage ratios are below their thresholds:

$$\rho_{p,c} < \theta_c \quad \wedge \quad \rho_{p,m} < \theta_m.$$

The default values $\theta_c = 0.80$ and $\theta_m = 0.80$ are configurable thresholds used to classify a Pod as busy relative to its effective requests. A ratio of 0.80 means that the observed usage has reached 80% of the configured request. It does not by itself indicate that the Pod is close to resource saturation because requests can represent different workload baselines.

The interpretation of the threshold therefore depends on request calibration and workload behavior. When a request equals its limit, a threshold of 0.80 also corresponds to 80% of the enforcement ceiling. When a request is lower than its limit, the same ratio indicates usage relative to the scheduling baseline but does not indicate proximity to the runtime ceiling. A workload with a small request and a large limit can frequently produce ratios above one. Thresholds greater than one are valid for intentionally bursty workloads. For example, a threshold of 10.0 blocks eviction when usage reaches ten times the request. Operators can adjust the threshold according to how requests represent the expected demand of workloads within the plugin's scope.

When any regular container lacks an effective request for a resource, the entire Pod-level dimension is marked as missing. The plugin does not divide aggregate usage by the requests of only the remaining containers because such a partial denominator would overstate usage relative to an incomplete baseline. Missing CPU and memory dimensions are evaluated independently using *missingRequestPolicy*.

With the default *Allow* policy, a missing dimension is skipped and contributes `busy = false`. The other resource dimension can still block eviction through the OR condition in the previously defined condition. With *Block*, a missing dimension contributes `busy = true` and blocks eviction. The default does not assert that

a Pod without requests is idle or that eviction is harmless. It indicates only that the plugin has no request-relative denominator from which to infer busyness. Operators can select *Block* when a conservative missing-request behavior is required.

3.3.3 Eviction Decision Workflow and Algorithm

Appendix B summarizes the Actual Usage Evictor decision workflow. The algorithm first checks whether the candidate Pod is inside the configured namespace and label scope. Pods outside the scope are allowed immediately because the plugin is not responsible for protecting them.

For scoped Pods, the plugin retrieves the latest Pod metrics from the metrics client and matches them with every regular container in the Pod specification. Missing or incomplete metrics block the eviction. For each resource, the plugin separately determines whether every container has an effective request. A complete dimension is aggregated and compared with its configured threshold. An incomplete request dimension is not divided by a partial aggregate request and is instead resolved using *missingRequestPolicy*. Eviction is blocked when either the CPU or memory dimension reports `busy = true`. Otherwise, the Pod passes the filter and the Descheduler may continue with the next pre-eviction filter or the final eviction request.

This workflow complements request-based strategies such as *ResourceDefragmentation*. A strategy may select a Pod because moving it improves cluster packing, but the Actual Usage Evictor can still reject the eviction when runtime metrics indicate that the Pod is actively consuming its requested resources.

Table 2. Actual Usage Evictor Plugin Arguments

Parameter	Type	Default	Purpose
<i>cpuUsageThreshold</i>	float	0.80	Blocks eviction when actual CPU usage divided by effective requested CPU reaches the configured threshold.
<i>memoryUsageThreshold</i>	float	0.80	Blocks eviction when actual memory usage divided by effective requested memory reaches the configured threshold.
<i>missingRequestPolicy</i>	string	<i>Allow</i>	Controls a resource dimension with a missing request. <i>Allow</i> skips the dimension; <i>Block</i> rejects the eviction.
<i>namespaces</i>	object	optional	Restricts the Pods evaluated by this plugin using namespace include or exclude rules.
<i>labelSelector</i>	object	optional	Restricts the Pods evaluated by this plugin using Kubernetes label matching.

3.3.4 Failure and Safety Behavior

The Actual Usage Evictor distinguishes unavailable runtime observations from absent declared request baselines. Runtime observations remain fail-closed: if the metrics collector or metrics client is unavailable, Pod metrics cannot be read, or any regular container has incomplete CPU or memory metrics, the eviction is blocked.

Missing requests follow *missingRequestPolicy* instead of the metrics failure path. This distinction is necessary because unavailable metrics leave the plugin unable to observe whether the Pod is busy, whereas a missing request leaves the plugin without a denominator for interpreting an otherwise available observation. The default *Allow* policy skips only the affected resource dimension; it does not change the fail-closed behavior for missing or incomplete metrics.

3.3.5 Plugin Configuration

```

1  apiVersion: descheduler/v1alpha2
2  kind: DeschedulerPolicy
3  profiles:
4    - name: default
5      pluginConfig:
6        - name: ActualUsageEvictor
7          args:
8            cpuUsageThreshold: 0.80
9            memoryUsageThreshold: 0.80
10           missingRequestPolicy: Allow
11           namespaces:
12             exclude:
13               - kube-system
14           plugins:
15             preEvictionFilter:
16               enabled:
17                 - ActualUsageEvictor

```

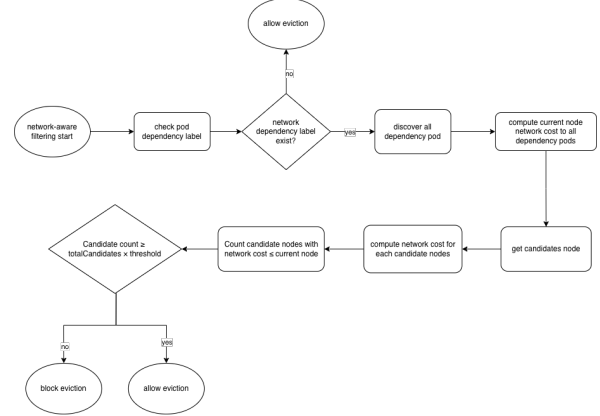


Figure 4. Network-Aware plugin high-level design.

Listing 2. Actual Usage Evictor configuration as a global pre-eviction filter.

3.4 Network-Aware Evictor Policy Design

We also propose the Network-Aware Descheduler policy to improve the pod eviction decisions in Kubernetes by considering the network cost factors. Different from the default descheduler that mainly considers the resource consumption, this policy considers the communication efficiency between the pods to avoid the performance deterioration caused by poor network locality.

The system has a two-layers plugin architecture. The first layer is a global PreEvictionFilter which is a safety device to prevent evictions that would increase the total network cost across all candidate nodes. The second layer is incorporated in node utilization strategies, offering a more accurate filtering mechanism by only analyzing the destination nodes selected by the strategy.

3.4.1 System Overview

The overall network-aware filtering system of the plugin is presented in Figure 4. When an eviction candidate is evalu-

ated, the process operates as a sequence of decision gates through the following steps:

First, the plugin inspects the candidate pod to determine if a network dependency label exists. If the pod has no defined dependencies, it is considered a network-unaware workload, and the plugin immediately allows the eviction since relocating it will not disrupt critical communication paths.

If a dependency label is found, the plugin proceeds to discover all related dependency pods currently running in the cluster. It then calculates the baseline network cost by computing the communication overhead between the candidate pod's current node and all nodes hosting its dependencies.

Next, the plugin retrieves a set of potential candidate destination nodes. Depending on the deployment configuration, these candidates can either be all schedulable nodes in the cluster (when operating as a global *PreEvictionFilter*) or a specific subset of overutilized or underutilized nodes gathered from a node utilization strategy. For each of these candidate nodes, it computes the hypothetical network cost that would occur if the pod were rescheduled to that location. The algorithm then counts the number of candidate nodes whose resulting network cost is less than or equal to the baseline cost of the current node.

Finally, the eviction decision is made by comparing this count against a threshold requirement. If the number of equal or better candidate nodes is greater than or equal to the total number of candidates multiplied by a configured threshold percentage ($\text{Candidate count} \geq \text{totalCandidates} \times \text{threshold}$), the eviction is allowed. If this condition is not met, indicating that a relocation would likely degrade network performance, the eviction is blocked.

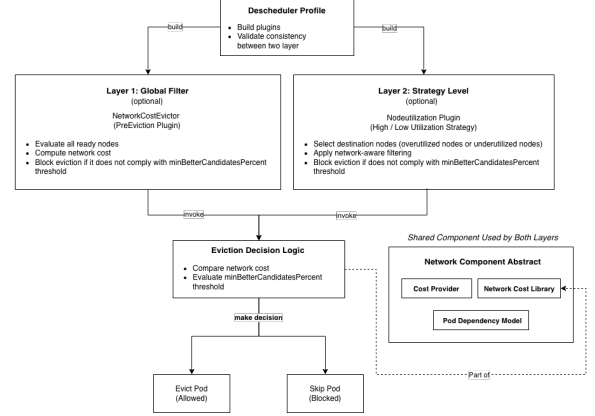


Figure 5. Network-Aware plugin architecture design.

3.4.2 Architecture Design

The detailed architecture of the Network-Aware Descheduler plugin is shown in Figure 5 based on the system overview described in Section 3.4.1. This architecture enhances the descheduler eviction process with a common network-aware assessment scheme that is uniformly applied to the architectural levels.

To support this design, the system is structured around three core components. The first component is the Network Cost Model abstracted into Cost Provider, which describes the calculation of the network distance between nodes. The abstraction enables topology-aware estimation based on node locality like region and zone, and latency-aware estimation based on real-time data collected via Prometheus.

The second component is the Pod Dependency Model, which enables the system to estimate the cost of communication considering interdependent workloads. This is done by a labeling method, where pods with the same configuration label (by default *network-group*) value are grouped together and regarded as a close dependency in the cost calculation.

The third component is Shared Network Cost Library, the centralized imple-

mentation for cost computation and the eviction decision maker. Both architectural layers use this library to ensure that the same logic is used to analyze network cost, regardless of the scope of evaluation.

The eviction decision itself is based on a common evaluation mechanism. The candidate nodes set is different between layers (from all scheduleable nodes to strategy selected subsets), but the same decision mechanism is utilized.

To balance between network latency and resource defragmentation goals, an eviction is allowed only if a given ratio of candidate nodes, which is specified by a configurable threshold, have a network cost lower or equal to ($\leq$) the current placement, otherwise the eviction is blocked. This threshold-based solution improves the likelihood of the following scheduling decisions to reschedule pods on nodes that do not degrade the network efficiency and still allow for appropriate workload redistribution.

3.4.3 Network Cost Model

To accommodate various deployment situations and accuracy levels, we design our Network Cost Model to have two distinct methodologies for cost computation: a topology-based cost model and a latency-based cost model. Both methodologies are encompassed inside the Cost Provider abstraction and can be chosen via configuration, as detailed in Section 3.4.2. The topology-based model offers a lightweight and static assessment, whereas the latency-based model facilitates a dynamic and measurement-driven evaluation grounded in real-time network conditions.

Topology Base Cost Model The topology-based cost model calculates the network cost based on the node locality information, e.g., zone and region information.

This is based on the assumption that the network latency rises with the physical or logical distance of nodes. Thus, the connection between nodes of the same zone is considered cheaper than the communication across zones or regions.

The model assigns specified costs based on relative position of nodes. A typical arrangement defines three tiers of cost: same-zone, same-region, and cross-region connectivity. For example, the cost of communication between nodes in the same zone is the lowest while the cost of communication across areas is the largest. The model sets default cost numbers to reflect this hierarchy: 1 for same-zone, 5 for same-region, 10 for cross-region. These values can be customized through configuration.

The advantage of this methods is its low processing overhead and independence from external monitoring systems. However, it is based on the simplistic assumption that the network performance is directly proportional to the physical proximity. In practice, this assumption may not always hold, as network latency might be altered by dynamic factors such as node load, congestion, or infrastructure variations. Nodes within the same zone can sometimes have higher latency than nodes in separate regions.

However, the topology-based model is a useful baseline for network-aware scheduling when real-time latency information is not accessible or not needed.

Latency Base Cost Model The latency-based cost model computes the network cost according to the observed node-to-node communication performance, which can provide a more accurate representation of the network conditions by considering the real-time latency between nodes and can adapt to the dynamic changes in the cluster environment by capturing tem-

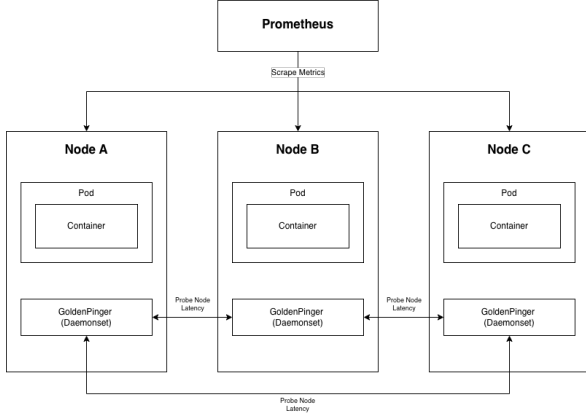


Figure 6. Prometheus scraping architecture for inter-node latency metrics via GoldenPinger DaemonSets.

porary factors like network congestion or unequal workload allocation, which may dramatically affect the communication performance.

To support this capability the system is integrated with a Prometheus based monitoring infrastructure that provides the latency metrics. The user must provide a Prometheus endpoint and a query for fetching node-to-node latency details. Latency numbers are usually derived from aggregated data across a certain time window (e.g. the average response time between nodes).

As shown in Figure 6, Goldenpinger is employed as the probing tool to test the node-to-node latency across the cluster. Each node runs an instance of Goldenpinger, which actively probes other nodes and exposes the resulting latency measurements. Prometheus then scrapes these parameters on a regular basis and stores them as time-series data. This version uses Goldenpinger, but any other tool that is able to collect latency data and expose it in a Prometheus-compatible format can be used.

Besides the query description, the configuration has to define the mapping between metric labels and cluster nodes. In particular, the latency measurements

are associated with the respective source and target node labels. This enables the system to create a delay matrix that specifies the communication cost between the nodes.

Consistency Between Layers Although each plugin exposes its own set of configurable parameters including threshold values, dependency labels, and cost model settings, specific elements related to network cost computation must remain aligned when both components are enabled within the same descheduler profile.

In particular, the selection of the cost evaluation approach must be consistent across both layers. When latency based cost evaluation is selected, both components are required to utilize identical Prometheus query definitions and consistent node label mappings, such as the source and target node identifiers. This ensures that network cost is derived from the same underlying observations and measurement semantics. Similarly, the dependency identification mechanism, defined through the *network-group* label, must be consistently configured to guarantee uniform grouping of interdependent pods during cost evaluation.

An exception to this requirement is the threshold parameter (e.g., *minBetterCandidatesPercent*), which may be configured independently for each component. This allows different levels of strictness to be applied at the global filtering stage and the strategy-level evaluation.

If any inconsistency is detected in the cost model configuration, the system performs validation at the profile level and raises an error. This design ensures that both components operate within a consistent cost model, thereby preventing discrepancies in cost evaluation that could otherwise lead to conflicting eviction de-

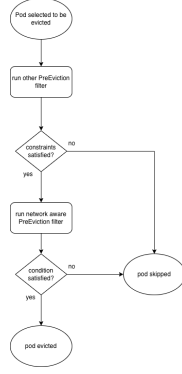


Figure 7. PreEviction filter chaining.

cisions. The corresponding configuration parameters are presented in Section 3.4.5.

3.4.4 Eviction Decision Workflow and Algorithm

The network-aware eviction decision workflow is implemented within the chaining of descheduler pre-eviction plugins. As shown in Figure 7, each candidate pod for eviction undergoes a chain of filtering processes before the actual eviction.

The first stage is the built-in *DefaultEvictor* plugin provided by the Kubernetes descheduler framework. This component evaluates several default eviction constraints such as pod state validation, scheduling constraints and node fit requirements to ensure that the pod is eligible for eviction (Kubernetes SIG Scheduling, nd). If any of these constraints is violated the eviction procedure is cancelled and the pod is skipped. Otherwise the eviction candidate continues to the next phase.

After passing the default pre-eviction filtering stage, our proposed Network-Aware Evictor plugin conducts an additional network-aware evaluation, as seen in Appendix C. This stage determines whether the eviction is expected to preserve or improve the efficiency of network connectivity in the cluster.

During the network-aware evaluation phase, the system first initializes

the network cost provider used for communication cost computation. If latency metrics are configured, the plugin queries Prometheus to obtain node-to-node latency measurements and constructs a latency-based cost provider. Otherwise, the system falls back to the topology-based cost provider, which estimates communication cost using predefined topology distances such as zone and region locality.

When the strategy-level component is enabled, the system additionally classifies cluster nodes according to their utilization state. Resource utilization is computed by comparing the total requested resources on each node against the allocatable node capacity. Nodes whose utilization exceeds the configured target threshold are classified as overutilized nodes, whereas nodes below the lower utilization threshold are classified as underutilized nodes. Depending on the selected balancing strategy, the candidate destination set is then derived from either the overutilized or underutilized node group. In contrast, the global filtering layer evaluates all scheduleable nodes in the cluster as potential destination candidates.

For each selected eviction candidate pod, the algorithm first checks whether the pod contains the configured dependency label. If no dependency label is present, the eviction is immediately permitted because no communication dependency needs to be preserved. Otherwise, the system identifies all dependent pods belonging to the same communication group using the *FindDependencyPods* function shown in Listing 3.

```

1  Function: FindDependencyPods
2  -----
3  Input:
4      pod                // the source pod being
   ⇨ evaluated
5      labelKey           // the network group label
   ⇨ key (e.g., "network-group")
6      nodes              // list of all cluster nodes
7      excludeSameOwner  // boolean flag to skip
  
```

```

8  ↪ same-controller replicas
9  Output:
10   D // set of dependency pods
11
12   groupValue (*$←$*)
13   ↪ pod.labels[labelKey]
14   if groupValue is empty then
15     return (*$←$*)
16   end if
17
18   D (*$←$*) (*$←$*)
19   for each node n (*$←$*) nodes do
20     for each pod p on node n do
21       if p.uid = pod.uid then
22         continue
23       end if
24       if p.labels[labelKey] (*$←$*)
25         ↪ groupValue then
26           continue
27         end if
28         if excludeSameOwner (*$←$*)
29           ↪ HasSameOwner(pod, p) then
30             continue
31           end if
32           D (*$←$*) D (*$←$*) {p}
33         end for
34       end for
35     end for
36
37     return D
38
39 Function: HasSameOwner
40 -----
41 Input:
42   a, b // two pods
43 Output:
44   HasSameOwner // boolean, true if they share
45   ↪ at least one controller
46
47   for each ownerRef (*$←$*)
48     ↪ a.ownerReferences do
49       for each ownerRef (*$←$*)
50         ↪ b.ownerReferences do
51           if (*$←$*).uid = (*$←$*).uid
52             ↪ then
53               return true
54             end if
55           end for
56         end for
57       end for
58     end for
59   return false

```

Listing 3. Network-Aware algorithm:
FindDependencyPods and *HasSameOwner*
functions

The *FindDependencyPods* function discovers communication dependencies by examining all pods across cluster nodes and comparing their configured dependency labels. Pods sharing the same dependency label are considered part of the same communication group. During this process, the algorithm excludes the evaluated pod itself and may optionally

exclude pods managed by the same controller, such as replicas belonging to the same Deployment, StatefulSet, or ReplicaSet by evaluating the *HasSameOwner* helper function (also shown in Listing 3). This behavior is controlled through the *excludeSameOwner* configuration parameter. The ownership relationship is determined by comparing Kubernetes owner references between pods.

After dependency discovery, the algorithm computes the communication cost between the evaluated pod and all dependency pods using the *ComputePlacementCost* procedure shown in Listing 4. This procedure evaluates the communication cost for both the current node placement and all candidate destination nodes. The set of evaluated candidate nodes depends on the architectural layer being used. The global filtering layer evaluates all scheduleable nodes in the cluster, whereas the strategy-level layer evaluates only the nodes selected by the balancing strategy.

```

1  Function: ComputePlacementCost
2  -----
3  Input:
4   candidateNode // the node being evaluated as
5   ↪ a placement target
6   D // set of dependency pods
7   ↪ (from Algorithm 1)
8   nodesMap // mapping from node name ->
9   ↪ node object
10  costProvider // function Cost(nodeA, nodeB)
11  ↪ -> (*$←$*)
12
13 Output:
14   totalCost // sum of pairwise communication
15   ↪ costs
16
17   totalCost (*$←$*) 0
18   for each pod d (*$←$*) D do
19     depNode (*$←$*)
20     ↪ nodesMap[d.nodeName]
21     if depNode not found then
22       continue
23     end if
24
25     totalCost (*$←$*) totalCost +
26     ↪ Cost(candidateNode, depNode)
27   end for
28
29   return totalCost

```

Listing 4. Network-Aware algorithm:
ComputePlacementCost function

Given a candidate pod P residing on node n , the total communication cost is calculated as the summation of network costs between node n and the nodes hosting all dependency pods $d \in D$, formally defined as follows:

$$\text{Cost}(n, P) = \sum_{d \in D} \text{Network}(n, \text{Node}_d)$$

The *ComputePlacementCost* procedure iterates through all dependency pods and retrieves the node hosting each dependency pod. Using the configured cost provider, the algorithm computes the pairwise communication cost between the candidate destination node and each dependency node. The resulting costs are accumulated to produce the total placement cost for the evaluated node.

After computing the communication cost for the current placement and all candidate nodes, the algorithm evaluates whether the eviction satisfies the configured threshold condition. For each candidate node, the computed candidate cost is compared against the current placement cost. The system then counts the number of candidate nodes whose communication cost is lower than or equal to the current placement cost.

Let K denote the number of candidate Nodes whose communication cost is no worse than the current placement, M the number of total candidates, and $\rho_{\min}$ the configured *MinBetterCandidatesPercent*. The eviction decision is determined using the threshold condition defined in the equation below:

$$K \geq M\rho_{\min}.$$

An eviction is permitted only if the number of candidate nodes satisfying the condition exceeds the minimum threshold defined by *MinBetterCandidatesPercent*,

which defaults to 50%. Otherwise, the eviction is rejected.

This threshold-based mechanism is intended to balance workload redistribution objectives with communication efficiency. By requiring a sufficient proportion of candidate nodes to maintain or improve network cost, the algorithm increases the likelihood that the Kubernetes scheduler will subsequently place the evicted pod onto a node that preserves or improves network locality after rescheduling.

3.4.5 Plugin Configuration

The configuration structure of our proposed Network-Aware Descheduler policy consists of two independent configuration corresponding to the two layers architectural components discussed in Section 3.4.1 and 3.4.2. The first layer configures the global *NetworkCostEvictor* plugin operating at the *PreEvictionFilter* phases, while the second layer extends node utilization balancing strategies with network-aware evaluation capabilities.

```

1  - name: "NetworkCostEvictor"
2    args:
3      networkGroupLabelKey: "network-group"
4      ↪ # Default: "network-group"
5      minBetterCandidatesPercent: 50
6      ↪ # Default: 50, Range: 1-100
7      excludeSameOwner: true
8      ↪ # Default: true
9      latencyMetrics:
10       prometheus:
11         # PromQL query that returns a vector of
12         ↪ node-to-node latency
13         # values. Each result must have labels
14         ↪ identifying the source
15         # and target node names.
16         query: >-
17         ↪ avg_over_time(goldpinger_peers_response_time_s_sum[5m])
18         /
19         ↪ avg_over_time(goldpinger_peers_response_time_s_count[5m])
20         sourceNodeLabel: "goldpinger_host_name"
21         ↪ # Default: "goldpinger_host_name"
22         targetNodeLabel: "goldpinger_peer_name"
23         ↪ # Default: "goldpinger_peer_name"

```

Listing 5. Network-Aware Layer 1: Global *PreEvictionFilter* configuration

Listing 5 show the *NetworkCostEvictor* configuration that defines the behavior

of the global network-aware filtering mechanism. This component evaluates candidate pod evictions against all schedulable nodes in the cluster before the eviction is permitted. The configuration includes parameters for dependency identification, minimum threshold percentage, pods ownership filtering flag, and optional latency-based cost evaluation through Prometheus metrics.

The *networkGroupLabelKey* parameter specifies the label key used to identify communication groups between pods. Pods sharing the same label value are treated as interdependent workloads during network cost evaluation. The *minBetterCandidatesPercent* parameter controls the strictness of the global filtering stage by defining the minimum percentage of candidate nodes that must provide a lower or equal communication cost before eviction is allowed. The *excludeSameOwner* parameter determines whether replicas managed by the same Kubernetes controller are excluded from dependency discovery.

The optional *latencyMetrics* section enables latency-based communication cost evaluation using Prometheus metrics. The configuration defines the PromQL query used to retrieve node-to-node latency measurements together with the source and destination node label mappings required to construct the latency matrix used during cost evaluation.

```

1  - name: "LowNodeUtilization" # or
    ↪ "HighNodeUtilization"
2  args:
3    # Strategy thresholds
4    thresholds:
5      cpu: 20
6      memory: 20
7    targetThresholds:
8      cpu: 50
9      memory: 50
10
11  networkAware:
12    networkGroupLabelKey: "network-group"
    ↪ # Default: "network-group"
13    strategy: "latency"
14    minBetterCandidatesPercent: 50
    ↪ # Default: 50, Range: 1-100

```

```

15    excludeSameOwner: true
    ↪ # Default: true
16    latencyMetrics:
17      prometheus:
18        # PromQL query that returns a vector
    ↪ of node-to-node latency
19        # values. Each result must have labels
    ↪ identifying the source
20        # and target node names.
21        query: >-
22
    ↪ avg_over_time(goldpinger_peers_response_time_s_sum[5m])
23        /
24
    ↪ avg_over_time(goldpinger_peers_response_time_s_count[5m])
25        sourceNodeLabel:
    ↪ "goldpinger_host_name" # Default:
    ↪ "goldpinger_host_name"
26        targetNodeLabel:
    ↪ "goldpinger_peer_name" # Default:
    ↪ "goldpinger_peer_name"

```

Listing 6. Network-Aware Layer 2: NodeUtilization extended configuration

The second configuration 6 extends node utilization strategies such as *LowNodeUtilization* and *HighNodeUtilization* with network-aware filtering capabilities. In addition to the standard resource balancing parameters such as *thresholds* and *targetThresholds*, the strategy exposes an optional *networkAware* configuration block. When this block is removed, the strategy behaves identically to the default Kubernetes descheduler implementation without network-aware evaluation.

The *networkAware* section defines the network cost evaluation behavior used during the strategy-level eviction filtering stage. Similar to the global filtering layer, this configuration supports dependency identification, minimum threshold percentage, pods ownership filtering flag, and multiple cost evaluation approaches. Unlike the global filtering layer, however, the strategy-level component evaluates only the subset of candidate schedulable destination nodes selected by the balancing strategy rather than all schedulable nodes in the cluster.

The configuration supports two mutually exclusive cost evaluation modes: *topologyBase* and *latencyBase*. The topology-

based mode uses predefined locality weights without the need for any external monitoring infrastructure, whereas the latency-based mode derives communication cost from real-time Prometheus metrics. When latency-based evaluation is enabled, the configuration must additionally provide the Prometheus query and node label mappings required to retrieve node-to-node latency measurements.

As discussed in Section 3.4.3, several configuration parameters must remain consistent when both architectural layers are enabled at the same time within the same descheduler profile. In particular, the dependency label configuration, the selected cost evaluation model, the Prometheus query definition, and the source and destination node label mappings must be identical across both layers. This ensures that both components evaluate network cost using the same communication model and underlying latency data.

The only exception to this consistency requirement is the *minBetterCandidatesPercent* parameter, which may be configured differently for each layer. This enables the global filtering stage and the strategy-level filtering stage to apply different levels of strictness during eviction evaluation. The system validates these configuration constraints during profile initialization and rejects invalid configurations that could result in inconsistent network cost evaluation behaviour between architectural layers.

4. RESULTS AND DISCUSSION

This section evaluates the developed Descheduler policies. It compares *ResourceDefragmentation* with the upstream *HighNodeUtilization* strategy across five node reclamation and fragmentation scenarios, evaluates how the Actual Usage

Evictor protects a busy application Pod under multiple balancing strategies, and measures the latency-resource trade-off of the Network-Aware Evictor.

4.1 ResourceDefragmentation Evaluation

4.1.1 Experimental Setup

The experiment uses one control-plane Node and six worker Nodes. Each worker provides 2000 *m* allocatable CPU and approximately 811 *Mi* allocatable memory. Pause containers reserve declared requests while contributing negligible runtime load, allowing the experiment to control the request-space layout precisely.

The scheduler uses *MostAllocated* and *BalancedAllocation*. Both policies operate on identical initial workloads and namespace scope. System workloads are excluded from eviction. The drain threshold is 0.40, the target-utilization ceiling is 0.90, each scenario permits at most five Descheduler passes, and every policy-scenario cell is repeated with three seeds.

The two systems under comparison are:

- **HighNodeUtilization (HNU):** the upstream utilization-based baseline with CPU and memory thresholds of 40%.
- **ResourceDefragmentation (RD):** the proposed resource-only policy using request-based Node state, RII/FSI-based source ordering, *predictSchedulerTarget*, and projected target bin score as its eviction-candidate criterion.

4.1.2 Scenarios

S1 primarily tests Node reclamation. S2 and S4 test whether dimensional fragmentation can be reduced when a scalar

Table 3. ResourceDefragmentation Evaluation Scenarios

Scenario	Intent	Initial condition
S1	Pure node reclamation	Uniformly underutilized Nodes whose workloads can be packed onto fewer workers.
S2	Reducible stranding	Complementary CPU-heavy and memory-heavy Nodes with no clearly high-utilization target.
S3	Mixed objective	Underutilized drain Nodes coexist with already dense balanced Nodes.
S4	Hogs and jumbo Pod	Skewed resource hogs must be merged around a large workload.
S5	Heterogeneous drain	A drain Node contains differently shaped Pods that should land on complementary targets.

utilization baseline has no suitable action. S3 combines node reclamation and shape preservation. S5 isolates shape-aware eviction-candidate selection during a heterogeneous drain.

4.1.3 Evaluation Metrics

The success of each policy-scenario cell is assessed through five complementary metrics that jointly capture node reclamation, dimensional balance, residual schedulability, and operational cost. Every metric operates on request-space Node state, so the cluster layout is described entirely by per-Node resource requests rather than runtime telemetry. The fundamental quantity shared by all metrics is the per-dimension utilization fraction of a worker Node n :

$$u_{n,R} = \frac{R_{\text{req}}(n)}{R_{\text{alloc}}(n)}, \quad R \in \{\text{cpu}, \text{mem}\}$$

Where:

- $u_{n,R}$ is the request-based utilization fraction of Node n along dimension R , bounded in $[0, 1]$.
- $R_{\text{req}}(n)$ is the sum of resource R requested by all evictable experiment Pods assigned to Node n .
- $R_{\text{alloc}}(n)$ is the allocatable capacity of Node n for resource R (2000 *m* CPU and approximately 811 *Mi* memory per worker).

Active Node Count The Active Node Count (N_{active}) is the primary indicator of node reclamation. A worker Node is counted as active only when it hosts at least one evictable experiment workload; infrastructure namespaces such as *kube-system* are excluded and therefore never keep a Node active.

$$N_{\text{active}} = \sum_{n=1}^N a_n$$

$$a_n = \begin{cases} 1, & \text{if } \sum R_{\text{req}}(n) > 0 \\ 0, & \text{otherwise} \end{cases}$$

Where:

- N_{active} is the number of worker Nodes hosting evictable experiment workloads.
- a_n is the binary activity indicator for Node n .
- N is the total number of worker Nodes available in the scenario.

A reduction in N_{active} between the before- and after-snapshots indicates that one or more Nodes have been fully drained and become reclaimable for scale-down. This metric directly measures the consolidation objective and is reported as the before-and-after pair together with the number of Nodes reclaimed.

Resource Stranding Score Because defragmentation optimizes the geometric *shape* of utilization rather than its absolute spread, the cluster is evaluated through a stranding score that measures how far each Node deviates from per-dimension balance. The cluster stranding score S aggregates the absolute CPU-to-memory imbalance across all active Nodes:

$$S = \sum_{n \in N_{\text{active}}} |u_{n,\text{cpu}} - u_{n,\text{mem}}|.$$

Where:

- S is the cluster-wide resource stranding score.
- $u_{n,\text{cpu}}$ and $u_{n,\text{mem}}$ are the request-based CPU and memory utilization fractions of Node n .

A Node with a large gap between its CPU and memory fractions strands the under-used dimension, since the remaining capacity cannot be allocated to a balanced reference workload. A lower S therefore indicates that CPU and memory fractions are more tightly aligned across the cluster, unlocking trapped capacity. Unlike a global standard deviation, S is insensitive to intentional differences in absolute load between Nodes and reacts only to per-Node dimensional skew.

Average Active-Node Utilization To quantify packing density, the Average Active-Node Utilization ($\bar{U}$) reports the mean request-based load of the Nodes that remain active after descheduling, averaged over both resource dimensions:

$$\bar{U} = \frac{1}{N_{\text{active}}} \sum_{n: a_n=1} \frac{u_{n,\text{cpu}} + u_{n,\text{mem}}}{2} \times 100\%$$

Where:

- $\bar{U}$ is the average utilization of the surviving active Nodes, expressed as a percentage.
- N_{active} is the number of active Nodes after descheduling.
- $\frac{u_{n,\text{cpu}} + u_{n,\text{mem}}}{2}$ is the mean per-dimension utilization fraction of Node n .

A higher $\bar{U}$ indicates that the surviving Nodes carry denser workloads, which is the expected consequence of successful consolidation. This metric complements N_{active} : reclaiming Nodes is only beneficial when the retained Nodes absorb the displaced load rather than leaving the cluster uniformly sparse.

Balanced Headroom The Balanced Headroom (H_{balanced}) converts residual request space into a schedulability-oriented quantity. It counts how many reference Pods, each requesting $200\text{ }m$ CPU and $80\text{ }Mi$ memory, can still be placed across the cluster after descheduling. Let $A_{\text{cpu}}(n)$ and $A_{\text{mem}}(n)$ denote the remaining requested CPU and memory space on Node n :

$$H_{\text{balanced}} = \sum_{n=1}^N \min \left(\left\lfloor \frac{A_{\text{cpu}}(n)}{c_{\text{ref}}} \right\rfloor, \left\lfloor \frac{A_{\text{mem}}(n)}{m_{\text{ref}}} \right\rfloor \right)$$

Where:

- H_{balanced} is the total number of $200\text{ }m/80\text{ }Mi$ reference Pods that fit in the remaining request space.
- $c_{\text{ref}} = 200\text{ }m$ and $m_{\text{ref}} = 80\text{ }Mi$ are the CPU and memory requests of the balanced reference Pod.
- the per-Node $\min(\cdot)$ enforces that a reference Pod fits only when both dimensions admit it, penalizing Nodes whose free capacity is stranded on a single axis.

A higher H_{balanced} indicates that defragmentation has reassembled fragmented free capacity into contiguous, jointly schedulable space. Because the per-Node term is limited by the tighter

dimension, a Node with abundant free CPU but little free memory contributes little headroom, which is precisely the stranding that the policy aims to remove.

The same construction generalizes to any reference Pod shape $r = (c_r, m_r)$, giving a family of probes H_r that reveal *which* kind of capacity a layout can still admit:

$$H_r = \sum_{n=1}^N \min \left(\left\lfloor \frac{A_{\text{cpu}}(n)}{c_r} \right\rfloor, \left\lfloor \frac{A_{\text{mem}}(n)}{m_r} \right\rfloor \right)$$

In addition to the balanced probe, three skewed probes are evaluated: a CPU-skewed Pod ($H_{\text{cpu-skewed}}$), a memory-skewed Pod ($H_{\text{mem-skewed}}$), and a large jumbo Pod ($H_{\text{jumbo}}, 800m/324Mi$). The balanced probe measures general schedulability, the two skewed probes measure how much single-axis capacity remains usable, and the jumbo probe measures whether the layout retains room for large indivisible workloads. Together they show that defragmentation does not merely free aggregate space but free space of the right shape.

Eviction Count and Efficiency The Eviction Count (E_{count}) measures the operational cost of a descheduling sweep as the total number of Pods relocated, extracted from the Descheduler job log:

$$E_{\text{count}} = \sum_{j \in J} e_j$$

Where:

- E_{count} is the total number of Pods evicted across all passes of a sweep.
- J is the set of log entries recorded during the job lifecycle.

Table 4. Active Worker Nodes Before and After Descheduling

Scenario	HNU	Reclaimed	RD	Reclaimed
S1	6 → 6	0	6 → 2	4
S2	6 → 6	0	6 → 3	3
S3	6 → 4	2	6 → 5	1
S4	5 → 5	0	5 → 4	1
S5	4 → 3	1	4 → 3	1

- e_j is a binary flag equal to 1 when log entry j records an eviction, and 0 otherwise.

Efficiency is interpreted jointly from E_{count} , the number of Nodes reclaimed (ΔN_{active}), and the reduction in stranding (ΔS). A policy is more efficient when it achieves the same structural improvement, that is, the same reclamation and balance gain, with fewer Pod movements, since every eviction imposes disruption on the running workload.

Convergence and Safety Finally, each run is validated for convergence and safety. The per-pass eviction count must eventually reach zero within the bounded loop of at most five passes, confirming that the policy reaches a stable fixed point rather than oscillating. After each run, every Pod must remain schedulable, so no after-snapshot may contain a Pending Pod, and no eviction request may remain stuck beyond the evicted Pods recorded in the log. These conditions verify that the consolidation pressure is exercised without violating feasibility or workload availability.

4.1.4 Node Reclamation

ResourceDefragmentation reclaims four Nodes in S1 and three in S2, while HNU performs no eviction in either scenario, as shown in Figure 8. HNU requires Nodes above its threshold to act as packing targets; uniformly low or uniformly

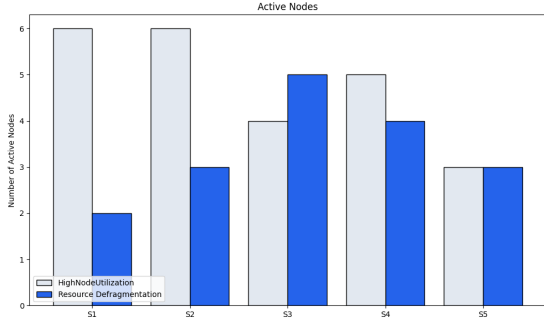


Figure 8. Active worker Nodes per scenario after descheduling.

fragmented layouts therefore produce no action. ResourceDefragmentation instead treats sparse Nodes and low bin-quality Nodes as resource-only consolidation sources, then uses *predictSchedulerTarget* to choose Pods with feasible denser targets even when the baseline cannot form high- and low-utilization groups.

S3 is the only reclamation case won by HNU. Dense balanced targets already exist, allowing the threshold policy to drain two Nodes greedily. ResourceDefragmentation respects the 0.90 ceiling and reclaims one Node. S5 is especially useful because both policies reclaim the same Node, allowing placement quality and eviction cost to distinguish them.

The consolidation effect is also visible in the density of the surviving Nodes. Figure 9 reports the average active-Node utilization $\bar{U}$ after descheduling: in the consolidation and fragmentation scenarios ResourceDefragmentation drives the retained Nodes to substantially higher density than HNU, confirming that the reclaimed capacity is absorbed rather than left uniformly sparse.

4.1.5 Stranding Reduction

As summarized in Figure 10, the result separates node reclamation from dimensional defragmentation. S1 begins with little stranding, so packing four Nodes into two leaves S essentially unchanged. In S2

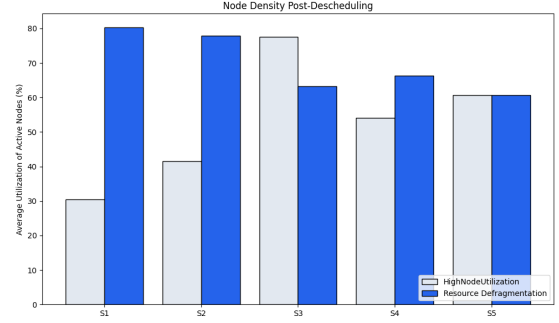


Figure 9. Average utilization of active Nodes after descheduling ($\bar{U}$).

Table 5. Cluster Stranding Score Before and After Descheduling

Scenario	HighNodeUtilization	ResourceDefragmentation
S1	0.049 → 0.049	0.049 → 0.050
S2	3.727 → 3.727	3.727 → 0.971
S3	1.395 → 1.408	1.395 → 1.386
S4	3.243 → 3.243	3.243 → 1.112
S5	1.338 → 1.298	1.338 → 0.123

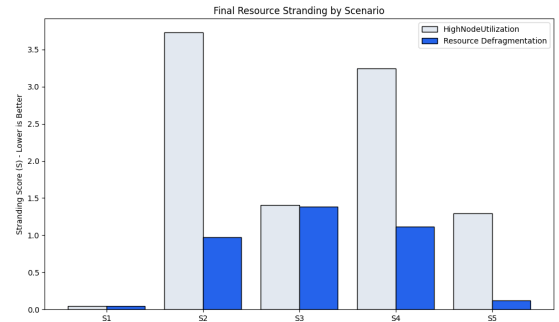


Figure 10. Final cluster stranding score S per scenario (lower is better).

and S4, ResourceDefragmentation uses the predicted-target score to combine complementary shapes and reduces S by approximately 74% and 66%, respectively; HNU does not act.

In S3, HNU reclaims one additional Node but slightly worsens S , whereas ResourceDefragmentation reduces it from 1.395 to 1.386. This shows the consequence of retaining resource shape in the target objective rather than optimizing Node count alone.

4.1.6 S5 Heterogeneous-Drain Analysis

S5 tests whether the selector distinguishes Pods on the same source Node by the shape of their predicted destinations. The source contains CPU-heavy and memory-heavy candidates, while other active Nodes expose complementary residual capacity.

ResourceDefragmentation predicts a feasible target for each candidate and compares projected target bin scores after applying the ceiling and pack-upward guards. It moves the CPU-shaped Pod to the Node with spare CPU-compatible balance and the memory-shaped Pod to the complementary Node. The source becomes empty after two evictions, and the receiving Nodes end in nearly balanced states. Consequently, S falls from 1.338 to 0.123, a reduction of approximately 91%.

HNU also empties one Node, but it requires three evictions and only reduces S to 1.298. Thus, equal Node reclamation does not imply equal defragmentation quality. The predicted-target criterion produces both a better placement and lower disruption.

4.1.7 Headroom and Eviction Efficiency

HNU retains higher distributed headroom in S1 because it leaves all six Nodes active; this is not a node reclamation success. In the fragmentation scenarios, ResourceDe-

Table 6. Balanced Headroom and Eviction Count

Scenario	HNU $H_{balanced}$	HNU Evict.	RD $H_{balanced}$	RD Evict.
S1	42	0	39	10
S2	15	0	29	6
S3	18	4	19	2
S4	13	0	24	5
S5	32	3	38	2

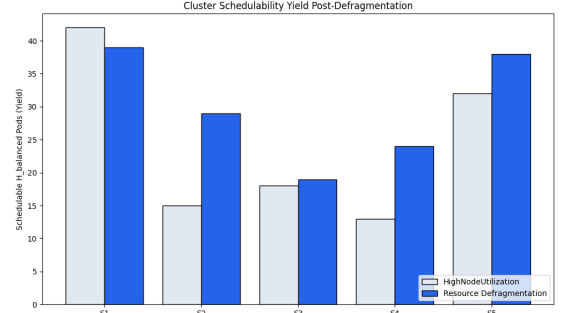


Figure 11. Balanced headroom $H_{balanced}$ per scenario after descheduling.

fragmentation exposes substantially more balanced headroom, as shown in Figure 11. S5 is the strongest efficiency result: two ResourceDefragmentation evictions reclaim one Node, reduce S by 1.215, and leave headroom for 38 balanced reference Pods. HNU uses three evictions for the same reclamation, reduces S by only 0.040, and leaves headroom for 32 reference Pods.

The eviction cost behind these gains is reported in Figure 12. ResourceDefragmentation spends evictions only where consolidation or defragmentation is achievable, and in S3 and S5 it reaches an equal or better cluster state than HNU with fewer Pod movements.

4.1.8 Shape-Specific Schedulability Yield

Balanced headroom alone does not show *which* kind of workload a defragmented layout can still admit. To make this explicit, the residual capacity is also probed with three skewed reference Pods: a CPU-skewed Pod, a memory-skewed Pod, and a large jumbo Pod (800 m/324 Mi).

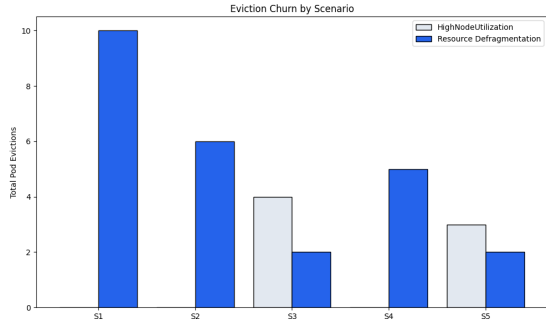


Figure 12. Total Pod evictions per scenario (E_{count}).

Table 7. Schedulability Yield by Reference-Pod Shape After Descheduling

Scenario	CPU-skewed		Memory-skewed		Jumbo	
	HNU	RD	HNU	RD	HNU	RD
S1	12	13	12	13	6	8
S2	9	9	9	11	0	6
S3	6	5	6	9	4	2
S4	4	8	4	7	3	4
S5	12	11	11	11	6	7

Table 7 reports the resulting yields after descheduling.

In the fragmentation scenarios, ResourceDefragmentation increases the yield for skewed and jumbo workloads where it matters most. In S2 and S4, where complementary CPU-heavy and memory-heavy Nodes are merged, it lifts memory-skewed yield ($9 \rightarrow 11$ in S2, $4 \rightarrow 7$ in S4) and restores jumbo capacity that the baseline cannot provide at all in S2 ($0 \rightarrow 6$), as shown in Figures 13, 14, and 15. In S5 it matches or slightly exceeds the baseline on every shape while using fewer evictions.

The two cases where HNU leads, CPU-skewed and jumbo yield in S3, reflect the same trade-off seen in the balanced headroom and stranding results: HNU empties an additional Node by greedily packing onto pre-dense targets, which leaves more whole-Pod slots free but at a slightly worse stranding score. ResourceDefragmentation instead preserves resource shape, so its remaining capacity

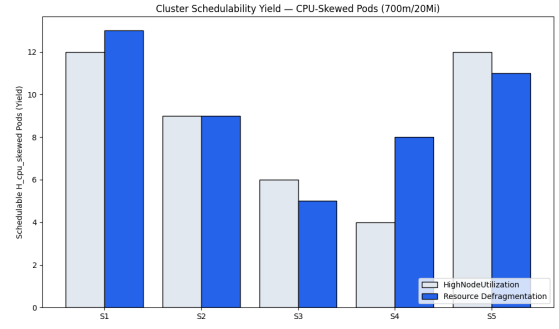


Figure 13. CPU-skewed schedulability yield $H_{cpu-skewed}$ per scenario after descheduling.

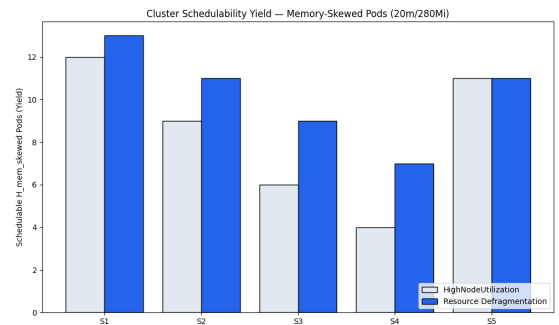


Figure 14. Memory-skewed schedulability yield $H_{mem-skewed}$ per scenario after descheduling.

is more evenly balanced across dimensions rather than concentrated on one emptied Node.

4.1.9 Convergence and Safety

Every scenario converges to a zero-eviction pass within two or three passes, with no oscillation. Across all repetitions, no after-snapshot contains a Pending Pod. All evicted Pods are controller-owned and eligible under the default evictor, infrastructure namespaces are excluded, and no eviction request remains outstanding.

Table 8. Representative Per-Pass Eviction Counts

Scenario	HighNodeUtilization	ResourceDefragmentation
S1	0	8, 2, 0
S2	0	6, 0
S3	4, 0	2, 0
S4	0	4, 1, 0
S5	3, 0	2, 0

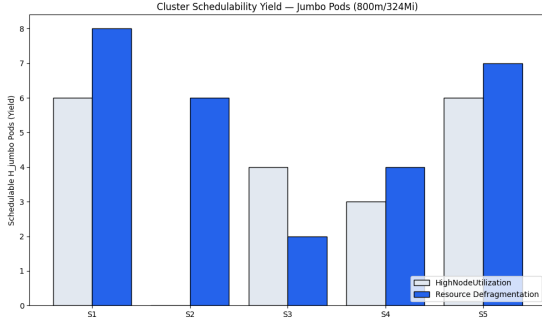


Figure 15. Jumbo (800 m/324 Mi) schedulability yield H_{jumbo} per scenario after descheduling.

These observations validate the feasibility guard, target ceiling, and bounded execution loop.

4.1.10 Resource Policy Findings

The evaluation yields four main findings:

- ResourceDefragmentation consolidates uniformly underutilized and uniformly fragmented layouts in which HNU is a no-op.
- ResourceDefragmentation reduces dimensional stranding in S2, S4, and S5 while preserving or increasing balanced schedulability headroom.
- Predicted-target bin-score selection is especially effective for heterogeneous drain Nodes, reducing S5 stranding from 1.338 to 0.123 with two evictions.
- The policy converges without leaving Pods Pending, showing that node reclamation pressure can be combined with explicit feasibility and utilization guards.

4.2 Actual Usage Evictor Evaluation

This section evaluates whether the Actual Usage Evictor can protect a currently busy application Pod across different Descheduler balancing objectives. Three

control-treatment pairs are evaluated under sustained API traffic. Each control enables a balancing strategy with the Default Evictor, while the corresponding treatment adds the Actual Usage Evictor as a global pre-eviction filter. The filter is expected to reject the busy API Pod while continuing to permit idle candidates.

The primary evaluation concerns application availability. It measures whether protecting the busy Pod reduces failed requests and service disruption following the Descheduler event. The cluster-side effect is evaluated according to the objective of each strategy. *ResourceDefragmentation* is assessed through stranding reduction, *HighNodeUtilization* through active-Node reduction, and *LowNodeUtilization* through normalized CPU and memory request-deviation reduction. This design distinguishes the common application-protection effect from the strategy-specific cost of excluding a busy eviction candidate.

4.2.1 Evaluation Scope

The evaluation uses controlled scenarios for *ResourceDefragmentation*, *HighNodeUtilization*, and *LowNodeUtilization*. Each scenario is designed to place a busy API Pod among the eviction candidates while preserving the placement objective of the evaluated strategy. The results therefore characterize the behavior of the Actual Usage Evictor in these controlled layouts rather than every possible cluster layout or workload composition.

Application availability is evaluated primarily through HTTP failure rate and service disruption duration. Successful-request tail latency is reported as a secondary metric because transient variation is present in the individual runs. The configured usage threshold of 0.80 is used to evaluate whether request-relative runtime filtering changes the eviction out-

come. The experiment does not attempt to determine an optimal threshold value.

4.2.2 Experimental Design

Common Workload and Load Generation Every comparison uses six worker Nodes and a singleton API Pod exposed through a dedicated NodePort Service. A *k6* constant-arrival-rate workload sends approximately eight requests per second only to this Service. Other experiment Pods do not receive foreground traffic. Each request performs fixed CPU work in the API container, whose CPU request is 50 *m* and CPU limit is 1000 *m*.

The Descheduler is triggered only after two consecutive Metrics API samples show that the API Pod’s CPU usage exceeds the configured ratio of 0.80. The decision-time ratios are 11.98 in R1, approximately 10.02 in H1, and approximately 10.10 in L1. These values confirm that the candidate is busy relative to its request. The CPU limit allows the container to burst above 50 *m*, but it is not a denominator in the Actual Usage Evictor decision.

Application results are normalized to the first 30 seconds after the recorded Descheduler event. R0 and R1 use a 30-second post-event capture. The longer HNU and LNU captures are truncated to the same event-aligned interval. Failure-rate denominators therefore contain only requests completed within this common window.

Experiment Execution Procedure Each experiment run follows a common ten-step procedure. The steps are identical across all six systems; only the Descheduler policy file and the raw capture duration differ.

1. **Layout deployment.** The strategy-specific layout is deployed and validated (Section 4.2.2).

2. **Before snapshot.** Node allocatable resources, Pod placements, and cluster metrics (S , H , N_{active}) are recorded.
3. **Load start.** A *k6* constant-arrival-rate generator and a Pod lifecycle watcher begin recording concurrently.
4. **Stabilization.** The system waits 30 *s* for the load to reach steady state.
5. **Threshold gate.** The Kubernetes Metrics API is polled every 10 *s* until two consecutive samples confirm the API Pod’s CPU usage-to-request ratio exceeds 0.80.
6. **Event timestamp.** The current time is persisted as t_{event} .
7. **Descheduler execution.** The Descheduler runs as a single-shot Kubernetes Job with the system’s policy.
8. **Post-event capture.** The script sleeps for the configured post-event duration (30 *s* for R0/R1, 45 *s* for H0/H1 and L0/L1) while *k6* and the lifecycle watcher continue recording.
9. **After snapshot.** The same cluster metrics are captured again.
10. **Shutdown and summarization.** Load generation is stopped and the *k6* JSON output is filtered to the 30 *s* event-aligned window for metric computation.

The post-event capture for HNU and LNU uses 45 *s* to ensure replacement Pods reach the Ready state before the after-snapshot. During analysis, only the first 30 *s* of *k6* data after t_{event} are retained so that failure-rate denominators are comparable across all six systems.

Table 9. Strategy-Specific Layouts for Actual Usage Evaluation

Pair	Initial layout	Intended strategy action
R0/R1	Complementary CPU- and memory-skewed Nodes. The API and an idle memory Pod share worker-6.	Move one source Pod to reduce request-space stranding.
H0/H1	Three dense destination Nodes and three underutilized one-Pod source Nodes. The busy API is isolated on worker-6.	Empty source Nodes and reduce the active-Node count.
L0/L1	Three underutilized destination Nodes and three overutilized source Nodes. The API and an idle fallback share worker-6.	Move one Pod from each source and reduce utilization deviation.

Strategy-Specific Cluster Layouts The three pairs use layouts that expose the placement objective of each balancing strategy, as summarized in Table 9.

HNU uses the *NodeResourcesFit* scheduler plugin with *MostAllocated* scoring so replacement Pods are packed onto the three destination Nodes. LNU uses the default *LeastAllocated* scheduler so evicted Pods are distributed to the underutilized Nodes. The ResourceDefragmentation pair uses the request-space layout described below.

ResourceDefragmentation Layout and Candidate Ordering The initial layout is a modified variant of the S2 reducible-stranding scenario. Three CPU-skewed worker Nodes, two memory-skewed worker Nodes, and one source Node carry the twelve experiment Pods listed in Table 10. The source Node (worker-6) is memory-skewed and hosts two Pods with the same CPU request but different memory requests. These are a user-facing API Pod and an idle memory-shaped worker Pod. This composition is the smallest layout in which *ResourceDefragmentationC2* deterministically selects the API Pod first while still leaving a structurally valid fallback candidate on the same Node.

The API Pod is exposed by a dedicated NodePort Service whose selector matches only the API Deployment. The memory Pod has no Service. Traffic is

Table 10. Initial Pod Placement for the Actual Usage Eviction Test

Node	Role	Pods	CPU frac	Mem frac	Traffic
worker-1	cpu-skewed	2 cpu Pods (750 m / 20 Mi each)	0.80	0.11	none
worker-2	cpu-skewed	2 cpu Pods (750 m / 20 Mi each)	0.80	0.11	none
worker-3	cpu-skewed	2 cpu Pods (750 m / 20 Mi each)	0.80	0.11	none
worker-4	mem-skewed	2 mem Pods (50 m / 240 Mi each)	0.10	0.65	none
worker-5	mem-skewed	2 mem Pods (50 m / 240 Mi each)	0.10	0.65	none
worker-6	source (mem)	1 API (50 m / 240 Mi) + 1 mem (50 m / 200 Mi)	0.10	0.60	API only

therefore routed deterministically to the API Pod and is independent of *kube-proxy* round-robin distribution. This layout satisfies three properties that are necessary for the experiment to be meaningful:

- Both source-Node Pods have identical CPU requests, so they appear equivalent under any CPU-only filter.
- The API Pod has a strictly larger memory request (240 Mi) than the memory Pod (200 Mi), so *ResourceDefragmentationC2* consistently selects the API Pod first when ranking by projected target bin score on a CPU-skewed destination.
- Only the API Pod receives traffic, so any disruption from evicting the API Pod is fully visible at the application boundary rather than being diluted across multiple replicas behind a single Service.

ResourceDefragmentationC2 predicts a CPU-skewed destination for each worker-6 candidate. The API Pod (50 m/240 Mi) produces a projected target bin score of 0.633, while the idle memory Pod (50 m/200 Mi) produces 0.596. The API Pod is therefore considered first. The Actual Usage Evictor does not modify this ranking because it operates at the *PreEvictionFilter* stage after the strategy has selected a candidate.

In HNU, the busy API occupies one of the three source Nodes that the strategy attempts to empty. Protecting the API

Table 11. Actual Usage Evictor Comparison Systems

Run	Strategy	Pre-eviction filters	Load
R0	ResourceDefragmentationC2	DefaultEvictor	k6 API load
R1	ResourceDefragmentationC2	DefaultEvictor + ActualUsageEvictor	k6 API load
H0	HighNodeUtilization	DefaultEvictor	k6 API load
H1	HighNodeUtilization	DefaultEvictor + ActualUsageEvictor	k6 API load
L0	LowNodeUtilization	DefaultEvictor	k6 API load
L1	LowNodeUtilization	DefaultEvictor + ActualUsageEvictor	k6 API load

therefore leaves worker-6 active. In LNU, the busy API is considered before an idle fallback on worker-6. Blocking the API allows the strategy to use the fallback and still complete one successful eviction from that source.

4.2.3 Comparison Systems

Each pair differs only in whether the Actual Usage Evictor is registered after the Default Evictor, as shown in Table 11.

The zero-suffix systems are controls without runtime safety filtering. The one-suffix systems use CPU and memory usage thresholds of 0.80. All evaluated Pods declare positive CPU and memory requests, so both dimensions have valid request-relative denominators.

4.2.4 Evaluation Metrics

The evaluation separates common application and eviction metrics from placement metrics that follow the objective of each strategy.

Common Application and Eviction Metrics The Actual Usage Block Count (B) records how many candidate evictions are rejected because at least one request-relative usage ratio reaches its threshold:

$$B = \sum_{j \in J} b_j$$

The HTTP Failure Rate (F) is the fraction of completed requests that return a non-2xx response within the common post-event window:

$$F = \frac{N_{\text{fail,post}}}{N_{\text{completed,post}}}.$$

Service Disruption Duration (D) measures the interval from the first failed request to the last failed request after the event. It is zero when no failed request is observed:

$$D = \begin{cases} t_{\text{last fail}} - t_{\text{first fail}}, & N_{\text{fail,post}} > 0, \\ 0, & N_{\text{fail,post}} = 0. \end{cases}$$

Successful-request p_{95} latency excludes failed responses. Failed HTTP 503 responses can complete faster than successful requests and can therefore make a mixed-response percentile appear lower during an outage. Latency is retained as a secondary metric because transient bursts appear before and after events. Eviction outcome records the successful eviction count, block count, evicted Pod identity, and whether the original API Pod remains.

Strategy-Specific Placement Metrics

ResourceDefragmentation uses positive stranding reduction ΔS and balanced-headroom change ΔH_b :

$$\Delta S = S_{\text{before}} - S_{\text{after}},$$

$$\Delta H_b = H_{\text{balanced,after}} - H_{\text{balanced,before}}.$$

HighNodeUtilization uses active-Node reduction ΔN_a :

$$\Delta N_a = N_{\text{active,before}} - N_{\text{active,after}}.$$

LowNodeUtilization uses the reduction in normalized request deviation for resource r :

$$\Delta \sigma_r = \sigma_{r,\text{before}} - \sigma_{r,\text{after}},$$

Table 12. Descheduler Behavior Under Sustained API Load

Metric	R0 (no plugin)	R1 (with plugin)
Evictions	1	1
Actual usage blocks (B)	0	1
Pod evicted	API (busy)	memory worker (idle)
Completed responses in 30 s	240	231
HTTP 503 responses	40	0
Failure rate F	16.67%	0.00%
Disruption duration D	4.875 s	0 s
Stranding reduction	0.816	0.000
Headroom change $\Delta H_{\text{balanced}}$	+4	+1

where each Node contributes its requested CPU or memory divided by its allocatable capacity. Positive values indicate improvement toward the corresponding strategy objective.

4.2.5 ResourceDefragmentation Results

Table 12 reports the ResourceDefragmentation pair. R0 evicts the busy API Pod and obtains the stronger defragmentation outcome. R1 blocks the API Pod, evicts the idle memory fallback, and preserves application availability.

Eviction Substitution and Application Protection In R0 the Descheduler issues exactly one eviction, targeting the API Pod on worker-6. The replacement Pod is admitted on worker-2, a CPU-skewed Node, which is consistent with the bin-score ranking computed in Section 4.2.2. In R1 the strategy still selects the API Pod first, but the Actual Usage Evictor observes 599 m CPU usage against a 50 m effective request, producing $\rho_{p,c} = 599/50 = 11.98$. Because this request-relative ratio exceeds the configured threshold of 0.80, the plugin rejects the eviction. The API Pod’s 1000 m CPU limit explains why it can burst far above its request without being capped at 50 m , but the limit does not participate in the ratio. The Descheduler then advances to the next candidate on the same source Node, the idle memory Pod, whose CPU and memory ratios are

Table 13. HighNodeUtilization Results

Metric	H0 (no plugin)	H1 (with plugin)
Successful evictions	3	2
Actual usage blocks (B)	0	1
Active Nodes	6 $\rightarrow$ 3	6 $\rightarrow$ 4
Active-Node reduction	3	2
Completed responses in 30 s	240	240
HTTP 503 responses	40	0
Failure rate F	16.67%	0.00%
Disruption duration D	4.836 s	0 s
Original API Pod remained	no	yes

both effectively zero, so the plugin allows the eviction. The replacement memory Pod is admitted on worker-4. In both configurations the source Node loses one Pod and the strategy still acts on worker-6. The identity of the evicted Pod is the only change.

The application-side effect is direct. During the normalized post-event window, R0 completes 240 requests and returns 40 HTTP 503 responses. This gives a failure rate of 16.67%, with failed responses spanning 4.875 s. R1 completes 231 requests with no failure and no disruption interval because the original API Pod remains on worker-6.

Defragmentation Trade-off The protection benefit reduces the ResourceDefragmentation objective. R0 moves the larger 240 Mi API Pod to a CPU-skewed Node. Stranding falls from 3.844 to 3.028, giving a reduction of 0.816, and balanced headroom increases by four reference Pods. R1 moves the 200 Mi idle fallback. Stranding remains 3.844, while balanced headroom increases by one. The strategy still performs a feasible movement, but it loses the portion of the defragmentation gain that depends on relocating the busy API Pod.

4.2.6 HighNodeUtilization Results

HNU classifies workers 4–6 as underutilized sources in both runs. H0 evicts all three source Pods, including the busy API

Table 14. LowNodeUtilization Results

Metric	L0 (no plugin)	L1 (with plugin)
Successful evictions	3	3
Actual usage blocks (B)	0	1
CPU deviation reduction	0	0
Memory deviation	0.21639 $\rightarrow$ 0.03091	0.21639 $\rightarrow$ 0.03091
Memory deviation reduction	0.18548 (85.7%)	0.18548 (85.7%)
Completed responses in 30 s	232	240
HTTP 503 responses	40	0
Failure rate F	17.24%	0.00%
Disruption duration D	4.873 s	0 s
Original API Pod remained	no	yes

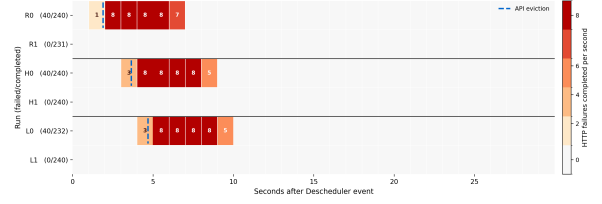
on worker-6. H1 allows the two idle source Pods but blocks the API at a CPU ratio of approximately 10.02. The filter therefore does not disable consolidation. It reduces the active-Node count from six to four instead of six to three.

H0 records 40 HTTP 503 responses and 4.836 s of disruption. H1 records no failure and no disruption. The observed trade-off is one retained active Node in exchange for preserving the busy API endpoint.

4.2.7 LowNodeUtilization Results

L0 evicts the busy API and one idle Pod from each of the other two sources. L1 blocks the API at a CPU ratio of approximately 10.10, then permits the idle fallback on worker-6. Both systems complete three successful evictions and produce the same final request distribution.

Memory request deviation falls from 0.21639 to 0.03091 in both runs, a reduction of 85.7%. CPU deviation remains 0.0125 because the layout uses memory requests to create the imbalance while candidate CPU requests are equal. L0 records 40 failed responses and 4.873 s of disruption. L1 records no failure and no disruption. In this scenario, runtime protection preserves the complete measured LNU balancing result.

**Figure 16.** Event-Aligned HTTP Failure Timeline and API Eviction Time over the Common 30-Second Post-Event Window

4.2.8 Cross-Strategy Discussion

Figure 16 shows the request failures using the same event-aligned 30-second post-event window for every run. Each cell counts HTTP 503 responses completed within a one-second interval. The dashed blue markers show the API eviction timestamps recorded in the Descheduler logs: 1.891 s in R0, 3.647 s in H0, and 4.680 s in L0. No marker appears in the treatment rows because the API eviction was blocked. Although the HNU and LNU experiments captured 45 seconds after the event, only the first 30 seconds are included for consistency with the ResourceDefragmentation experiments. Request timestamps represent completion time rather than generation time, so the number of completed responses can differ between runs despite the constant configured arrival rate of eight requests per second.

The three control runs exhibit compact failure intervals after the Descheduler event, whereas the corresponding Actual Usage Evictor treatments contain no failed response. Figure 17 summarizes the failure rates across all six systems. The control configurations consistently produce failure rates between 16.67% and 17.24%, while every treatment records 0%.

Figure 18 shows the corresponding service disruption durations. All control runs exhibit approximately 4.8 s of disruption, while every treatment records 0 s.

Table 15 separates this common ap-

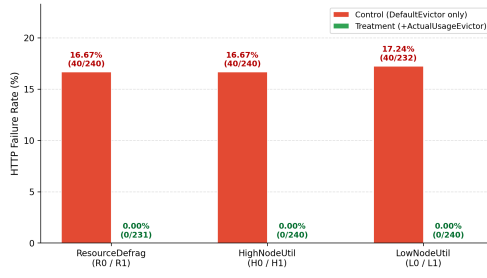


Figure 17. HTTP Failure Rate Comparison Across All Strategy Pairs

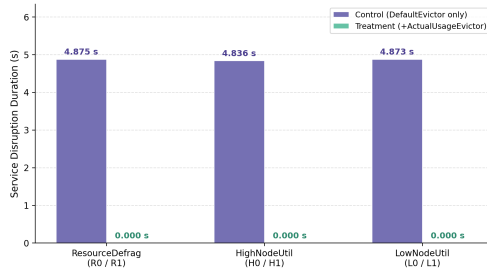


Figure 18. Service Disruption Duration Comparison Across All Strategy Pairs

plication effect from the strategy-specific placement cost. The failure-rate denominator is the number of requests completed within the common interval.

The Actual Usage Evictor consistently changes the decision for the busy API candidate while allowing idle candidates to pass. The placement cost depends on whether the remaining candidate can satisfy the active strategy. R1 retains movement but loses the strongest stranding improvement. H1 retains partial consolidation but cannot empty the protected source. L1 substitutes an idle Pod and

Table 15. Cross-Strategy Actual Usage Evictor Results

Pair	Availability result	Strategy-specific effect
R0/R1	Failure rate falls from 16.67% to 0%. Disruption falls from 4.875 s to 0 s.	R1 still evicts an idle fallback, but stranding reduction falls from 0.816 to 0, and balanced-headroom gain falls from four to one.
H0/H1	Failure rate falls from 16.67% to 0%. Disruption falls from 4.836 s to 0 s.	H1 still reclaims two source Nodes, but active-Node reduction falls from three to two.
L0/L1	Failure rate falls from 17.24% to 0%. Disruption falls from 4.873 s to 0 s.	L1 completes the same three evictions and retains the 85.7% memory-deviation reduction.

retains the complete measured balancing result.

4.2.9 Actual Usage Findings

The evaluation yields four main findings:

- The Actual Usage Evictor consistently blocked the busy API Pod while allowing idle candidates to remain eligible. The control configurations evicted the API Pod, whereas the corresponding treatment configurations preserved its original Pod instance.
- Protecting the API Pod eliminated the observed eviction-induced outage in all three loaded comparisons. Over the common 30 s post-event window, the control failure rates ranged from 16.67% to 17.24%, while every treatment recorded 0%. The observed disruption durations of approximately 4.8 s in the controls were reduced to 0 s in the treatments.
- The placement cost depended on the strategy and the available fallback candidate. ResourceDefragmentation retained an eviction but lost the stranding reduction produced by moving the API Pod. HighNodeUtilization reclaimed two rather than three source Nodes because the protected API kept one Node active. LowNodeUtilization substituted an idle Pod and retained the same three evictions and 85.7% memory-deviation reduction.
- Successful-request latency exhibited inconsistent bursts before and after the eviction event. Latency is therefore treated as a secondary metric, while failure rate and service disruption duration provide the primary evidence of application protection.

4.3 Latency and Defragmentation Trade-off Test

In this section we evaluate the trade-off between network latency preservation and resource defragmentation within a Kubernetes cluster using the proposed Network-Aware plugin. The evaluation focuses on how the plugin maintains communication efficiency by preventing pod evictions that could potentially increase network latency. The experiments are conducted using three balancing strategies, namely LowNodeUtilization, HighNodeUtilization, and custom ResourceDefragmentation, to analyze the impact of network-aware eviction filtering under different workload redistribution approaches.

4.3.1 Experimental Setup and Environment Configuration

To evaluate the effectiveness of the proposed Network-Aware Descheduler against resource fragmentation and degraded latency anomalies, we built a physical-to-logical hybrid testing sandbox. The design validates descheduling actions against varying node performance tiers under tight resource ceilings, providing deterministic telemetry data for evaluation.

Infrastructure and Cluster Topology

The experimental testing cluster is hosted inside an isolated Virtual Private Cloud (VPC) on Amazon Web Services (AWS). Due to the structural restrictions of the cloud sandbox environment, all instances are physically provisioned within the *us-east-1* geographic region. The infrastructure pool comprises five distinct *t3.micro* virtual machines acting as worker nodes, alongside a dedicated administrative control plane and one worker specific for Prometheus.

A critical engineering constraint of

the *t3.micro* hardware profile is its highly restricted footprint: each instance features 2 vCPUs and a bare 1 GiB (1024 MiB) of physical RAM. After accounting for vital system daemon overheads (including the Linux kernel, Kubelet, container runtime, and Flannel network plugins), the net allocatable memory per node drops to a constrained window of approximately 800 MiB.

To model real-world high-availability environments, the underlying physical cluster nodes are distributed across distinct Availability Zones (AZs) as follows:

- *us-east-1d*: worker-1, worker-2
- *us-east-1b*: worker-3
- *us-east-1a*: worker-4, worker-5

Network Profile and Topology Modeling

To evaluate the impact of network locality on scheduling and descheduling decisions, the cluster topology was configured to include nodes with heterogeneous network characteristics distributed across multiple logical availability zones:

- **Normal Network Conditions:** Consists of worker-1, worker-2, and worker-3. These nodes retain their native cloud-provider network performance. Communication within the same availability zone experiences approximately 0.3 ms latency, while communication across availability zones incurs a natural latency of approximately 1-2 ms.
- **Degraded Network Conditions:** Consists of worker-4 and worker-5, which are assigned to a separate logical availability zone and configured to emulate constrained network conditions. These nodes experience significantly higher communication latency than the rest of the cluster,

representing a lower-quality network domain within the topology.

To achieve the network degradation, the primary operating system network interface (*ens5*) on both worker-4 and worker-5 was targeted. A deterministic outbound network emulation queue discipline (*qdisc*) rule was applied directly to the root interface using the Linux Traffic Control (*tc*) framework:

```
sudo tc qdisc add dev ens5 root netem delay 80ms
↳ 5ms distribution normal
```

Crucially, because this traffic shaping rule acts as a uniform egress filter, it creates a multi-hop latency matrix depending on the communication path. When a normal node (worker-1) initiates a round-trip connection with a degraded node (worker-4), the packet triggers an 80 ms delay leaving worker-4's interface during the response phase. However, when worker-4 and worker-5 communicate with each other, both machines enforce their outbound egress rules. This creates an additive round-trip time (RTT) accumulation:

$$RTT_{T2 \leftrightarrow T2} = D_4 + D_5 \\ \approx 160 \text{ ms.}$$

Telemetry and Monitoring Architecture

To provide the network-aware descheduler with continuous data streams, we deployed a tight telemetry loop combining a full-mesh prober with a time-series database.

Bloomberg's Goldpinger (version 3.10.1) was deployed as a cluster-wide DaemonSet, pinning a network auditing pod to each of the five worker nodes. Every 30 seconds, each Goldpinger instance executes a cross-node internal web check to every discovered peer over port 8080. This creates an active, real-time matrix of the data paths between all five nodes.

To capture and aggregate this data, a minimal installation of Prometheus was deployed using kube-prometheus-stack Helm chart. Because of the strict memory limits of the *t3.micro* nodes, we limit the monitoring stack to prevent system crashes. High-overhead default dependencies including Grafana, Alertmanager, kube-state-metrics, and node-exporter were entirely disabled.

The Prometheus server instance was pinned to worker-6 (dedicated Prometheus node) using a targeted *nodeSelector*. Its operational resource limits were tightly bounded to prevent memory leakage from exhausting the node's approximately 800 MiB allocatable ceiling:

```
1 prometheusOperator:
2   resources:
3     limits: { memory: "200Mi" }
4     requests: { cpu: "20m", memory: "100Mi" }
5
6 prometheus:
7   prometheusSpec:
8     retention: "6h"
9     scrapeInterval: "45s"
10    resources:
11      limits: { memory: "450Mi" }
12      requests: { cpu: "50m", memory: "200Mi" }
```

Listing 7. Prometheus resources config for Latency and Defragmentation Trade-off Test

By expanding the global *scrapeInterval* to 45s and restricting data retention to a lean 6h window, the internal time-series indexing map remains small enough to sit safely in 800 MiB usable RAM. Telemetry target discovery is governed by a native PodMonitor custom resource deployed to the *prom* namespace. This monitor hooks directly into the named container exposed in the Goldpinger daemon manifest, automatically mapping the scraping loop to target port 8080.

The telemetry data is collected by evaluating two core histogram vectors: *goldpinger_peers_response_time_s_sum* and *goldpinger_peers_response_time_s_count*. The network-aware descheduler will

Strategy / Scenario	Low Node Util.	High Node Util.	Imbalance Index Util.
Baseline	✓	✓	✓
Topology L1	✓	✓	✓
Topology L2	✓	✓	—
Topology L1 & L2	✓	✓	—
Latency L1	✓	✓	✓
Latency L2	✓	✓	—
Latency L1 & L2	✓	✓	—

Table 16. Experimental Matrix Mapping for Latency and Defragmentation Trade-off Test

queries these metrics using an integrated PromQL engine over a rolling 5-minute mathematical average latency via the following unified query string, where G_{sum} and G_{count} denote those two Goldpinger vectors:

$$L_{\text{net}} = \frac{\text{rate}(G_{\text{sum}}[5\text{m}])}{\text{rate}(G_{\text{count}}[5\text{m}])}$$

The query utilizes host name as its source indicator and peer name as its target label mapping. This allows the descheduler to programmatically construct a real-time topology map of the cluster’s network characteristics.

4.3.2 Testing Scenario

The evaluation is split into three experimental testing scenario: LowNodeUtilization, HighNodeUtilization, and ResourceDefragmentation. For each scenario we test combination of strategies as shown in the experimental matrix defined below:

We also configure specific plugin configuration for each scenario as follows:

- **Low Node Utilization:** This scenario utilizes a strict resource boundary constraint where any node under 20% CPU and Memory usage is marked as underutilized, while any node exceeding 50% CPU and Memory usage is classified as overutilized.
- **High Node Utilization:** This scenario identify nodes with less than

30% CPU and 40% Memory resource utilization as underutilize node and nodes with greater usage in either metrics as overutilization node to support node downscaling.

- **Imbalance Index Utilization:** This scenario utilizes 0.15% Imbalance Index Threshold and limit eviction to 50 pods as node reclamation candidates to support node downscaling.

4.3.3 Workload Profiling and Initial Distribution

To evaluate the the network-aware descheduler, we constructed a standardized benchmarking workload. By establishing a consistent initial pod distribution across our test cluster, we isolate the descheduler’s decision-making logic and mathematically compare the performance of different eviction strategies under identical starting conditions.

The core of the evaluation revolves around a simulated e-commerce checkout flow containing network-sensitive microservices. The application is logically divided into three primary components:

- **Database (*db*):** Pinned to a specific healthy node to serve as the geographical "anchor" for network cost calculations.
- **Transaction Service (*transaction*):** A middle-tier service co-located with the database on the same physical node.
- **Payment Service (*payment*):** The scalable frontend component. These pods represent the critical workload that the descheduler must manage. Their placement relative to the *transaction* service directly dictates the application’s end-to-end latency.

Node	Zone	Pods	CPU Alloc.	Mem Alloc.	Status
worker-3	1b (Healthy)	1 db + 2 txn	~650m (36%)	~246Mi (30%)	Normal (in bounds)
worker-1	1d (Healthy)	1 pay + 5 unaware	~1000m (55%)	~304Mi (38%)	OVERUTILIZED (>50%)
worker-2	1d (Healthy)	1 pay + 5 unaware	~1000m (55%)	~304Mi (38%)	OVERUTILIZED (>50%)
worker-4	1a (Degraded)	0 pods	0m (0%)	0Mi (0%)	UNDERUTILIZED (<20%)
worker-5	1a (Degraded)	0 pods	0m (0%)	0Mi (0%)	UNDERUTILIZED (<20%)

Table 17. Resource Allocation: Initial State for LowNodeUtilization Strategy

All pods within this checkout flow are tagged with the **network-group: checkout-flow** label. To manipulate cluster utilization without increasing the footprint of the critical microservices, we also utilize generic network-unaware pods that consume CPU and Memory resources but without any network-group label attached to it.

By strategically deploying network-unaware pods alongside our critical *payment* services, we force the cluster into specific initial imbalance states tailored for each scenario.

LowNodeUtilization Scenario The *LowNodeUtilization* strategy resolves resource packing by spreading pods from overutilized nodes to underutilized ones. To evaluate this, we aggressively pack both critical *payment* pods and heavy network-unaware workloads onto a small subset of nodes, pushing their utilization well above the target thresholds. The remainder of the cluster (including the degraded zone) is left completely empty.

Table 17 details the exact state of the cluster prior to descheduler intervention.

This starting state tasks the descheduler with safely distributing the critical *payment* pods to the empty healthy nodes while strictly avoiding the empty degraded zone.

HighNodeUtilization Scenario Conversely, the *HighNodeUtilization* strategy is designed to pack a fragmented cluster by evicting pods from underutilized nodes and packing them onto overutilized nodes. We place the critical *payment* pods sparsely

Node	Zone	Pods	CPU Alloc.	Mem Alloc.	Status
worker-3	1b (Healthy)	1 db + 2 txn	~756m (37.8%)	~366Mi (45.7%)	OVERUTILIZED (>30%)
worker-4	1a (Degraded)	6 unaware	~1006m (50.3%)	~408Mi (51.0%)	OVERUTILIZED (>30%)
worker-5	1a (Degraded)	5 unaware	~856m (42.8%)	~360Mi (45.0%)	OVERUTILIZED (>30%)
worker-1	1d (Healthy)	1 pay + 1 unaware	~306m (15.3%)	~152Mi (19.0%)	UNDERUTILIZED (<30%)
worker-2	1d (Healthy)	1 pay + 1 unaware	~306m (15.3%)	~152Mi (19.0%)	UNDERUTILIZED (<30%)

Table 18. Resource Allocation: Initial State for HighNodeUtilization Strategy

across otherwise empty nodes. We then intentionally overload the remaining nodes exclusively with heavy network-unaware workloads, creating the heaviest "gravity wells" within the network-degraded zone.

Table 18 details the exact state of the cluster prior to node reclamation.

This starting state forces the descheduler to evict the *payment* pods to pack the cluster. The challenge is preventing the *kube-scheduler's* default *MostAllocated* strategy from blindly bin-packing these critical workloads into the overloaded, but network-degraded, nodes.

ResourceDefragmentation Scenario The *ResourceDefragmentation* scenario evaluates the descheduler's ability to correct multi-dimensional resource skew (CPU vs. Memory) without relying on strict utilization thresholds. Rather than simply packing or spreading pods, this strategy seeks to optimize the geometric "shape" of resource utilization on each node by matching CPU-heavy workloads with Memory-heavy environments, and vice versa.

To evaluate this behavior, we construct a highly fragmented cluster environment featuring massive orthogonal skews. We force the healthy nodes to become extremely CPU-heavy by packing them with the critical *payment* pods and CPU-intensive network-unaware workloads. Conversely, we force the network-degraded nodes to become extremely Memory-heavy by overloading them with memory-intensive network-unaware workloads while starving them of CPU.

Node	Zone	Pods	CPU Alloc.	Mem Alloc.	Status
worker-3	1b (Anchor)	1 db	~1506m (75.3%)	~520Mi (65.0%)	Normal (+0.103 RII)
worker-1	1d (Heavy Overhead)	1 pay + 2 unaware-cpu	~1805m (90.3%)	~416Mi (52.0%)	CPU SKEWED (+0.383 RII)
worker-2	1d (Healthy)	1 pay + 2 unaware-cpu	~1706m (85.3%)	~220Mi (27.5%)	CPU SKEWED (+0.578 RII)
worker-4	1a (Degraded)	2 unaware-mem	~206m (10.3%)	~670Mi (83.8%)	MEM SKEWED (-0.735 RII)
worker-5	1a (Degraded)	2 unaware-mem	~206m (10.3%)	~670Mi (83.8%)	MEM SKEWED (-0.735 RII)

Table 19. Resource Allocation: Initial State for ResourceDefragmentation Strategy

Table 19 details the exact state of the cluster prior to defragmentation.

This starting state presents a mathematical trap for the descheduler. From a purely resource-optimizing perspective, moving the CPU-heavy *payment* pods to the memory-heavy degraded nodes would perfectly resolve the cluster’s fragmentation. The challenge is ensuring the network-aware filter intercepts this mathematically optimal, but network-destructive, decision.

4.3.4 Evaluation Metrics

To quantitatively measure and compare performance trade-offs, and systemic impact of the developed Network-Aware Descheduler against the standard upstream baseline, we establish a multi-dimensional evaluation framework. The success of the various descheduling configurations is assessed using five core metric categories that systematically capture algorithmic behavior, cluster resource redistribution efficiency, network infrastructure locality, and structural topology alignment.

M1: Eviction Count The Eviction Count serves as a direct indicator of algorithmic activity and work node reclamation aggressiveness within a single execution sweep. It counts evicted pods from their original host nodes. Information for this metric is extracted straight from the single-shot job log by quantifying the total occurrence of specific target eviction notification strings:

$$E_{count} = \sum_{j \in J} e_j$$

Where:

- E_{count} is the total number of pods evicted during a specific descheduling sweep.
- J represents the set of log entries recorded during the execution job lifecycle.
- e_j is a binary flag equal to 1 if log entry j matches the string "Evicted pod", and 0 otherwise.

From an architectural standpoint, network-aware configurations are structurally expected to show a decreased eviction profile compared to the unrestricted baseline. This contraction representatively isolates the "cost of network-awareness," where the integrated cost filters purposely block rebalancing movements that would otherwise degrade inter-service network latency bounds. This metric is evaluated universally across all three testing scenarios (*LowNodeUtilization*, *HighNodeUtilization*, and *ResourceDefragmentation*) to quantify the baseline operational cost and activity level of each filter.

M2: Defragmentation Standard Deviation and Active Node Count To properly evaluate and distinguish between the opposing structural goals of load defragmentation and bin-packing, we create a standard framework to measure cluster resource standard deviation. The evaluation is performed independently across both CPU and memory resource dimensions, where the utilization coefficient $U_{n,R}$ for each worker node n and resource dimension $R \in \{\text{cpu}, \text{memory}\}$ is defined as follows:

$$U_{n,R} = \left(\frac{R_{\text{req}}(n)}{R_{\text{alloc}}(n)} \right) \times 100\%$$

Where:

- $U_{n,R}$ represents the request-based utilization percentage of node n for resource R .
- $R_{\text{req}}(n)$ denotes the total requested amount of resource R from all pods assigned to node n .
- $R_{\text{alloc}}(n)$ denotes the allocatable capacity of node n for resource R , as reported by the Kubelet API.

Using these utilization coefficients, the workload distribution variance across the cluster is modeled using the population standard deviation σ_R :

$$\sigma_R = \sqrt{\frac{1}{N} \sum_{n=1}^N (U_{n,R} - \mu_R)^2}$$

Where:

- σ_R is the population standard deviation of resource utilization for resource R .
- N represents the total number of worker nodes in the cluster ($N = 5$).
- μ_R is the mean utilization percentage for resource R computed across all nodes.

Simultaneously, we also track the Active Node Count (N_{active}). A node is classified as active only if it hosts non-DaemonSet application workloads, excluding immutable infrastructure services such as Goldpinger, kube-proxy, and Flannel that cannot be redistributed by the descheduler.

$$N_{\text{active}} = \sum_{n=1}^N a_n$$

$$a_n = \begin{cases} 1, & \text{if } \sum R_{\text{workload}}(n) > 0 \\ 0, & \text{otherwise} \end{cases}$$

Where:

- N_{active} is the total number of worker nodes running application workloads.
- a_n is the binary activity indicator for node n .
- $R_{\text{workload}}(n)$ represents the compute requests generated by non-DaemonSet workloads assigned to node n .

By combining σ_R and N_{active} , the framework evaluates whether the descheduling policy achieves its intended infrastructure objective. A low σ_R together with a high N_{active} indicates successful defragmentation, where workloads are distributed evenly across the cluster infrastructure. Contrary, a high σ_R combined with a low N_{active} indicates successful bin-packing, where workloads are packed onto a smaller subset of nodes while leaving other nodes idle and available for scale-down. An inefficient cluster state is characterized by both high σ_R and high N_{active} , indicating that workloads remain unevenly distributed while still occupying most of the infrastructure footprint. The σ_R metric is primarily utilized in the *LowNodeUtilization* and *HighNodeUtilization* scenarios to evaluate the absolute spread of cluster resources. Conversely, the Active Node Count (N_{active}) is specifically leveraged in the *HighNodeUtilization* scenario to measure bin-packing efficiency and node-reclamation capability.

M3: Average Cross-Service Latency To assess the operational impact of descheduling decisions on microservice communication pathways, the Average Cross-Service Latency is measured. This metric evaluates the communication cost between interacting components that belong to the same labeled network group. To avoid

bias from intra-deployment communication, the framework excludes pods managed by the same deployment owner, ensuring that only true cross-service dependencies are evaluated.

$$P_{\text{pairs}} = \left\{ (p_1, p_2) \mid \begin{array}{l} p_1 \in S_x, p_2 \in S_y, \\ S_x \neq S_y \end{array} \right\}$$

$$C_{\text{total}} = \sum_{(p_1, p_2) \in P_{\text{pairs}}} L(N(p_1), N(p_2))$$

$$L_{\text{avg}} = \frac{C_{\text{total}}}{|P_{\text{pairs}}|}$$

Where:

- P_{pairs} represents the set of all unique cross-service pod pairs belonging to the same network group.
- p_1 and p_2 are running pods belonging to different services (S_x and S_y) within the *checkout-flow* application group.
- C_{total} is the accumulated communication cost across all evaluated service-to-service connections.
- $L(a, b)$ represents the network latency between node a and node b , derived from a 5-minute rolling average collected through Goldpinger telemetry.
- $N(p)$ is a mapping function that returns the worker node hosting pod p .
- L_{avg} represents the average cross-service latency across the evaluated application mesh.

This is a universal metric applied across all scenarios (*LowNodeUtilization*, *HighNodeUtilization*, and *ResourceDefragmentation*) as the primary and most critical indicator of network performance preservation.

M4: Network Degraded Pod Count The final validation metric for this test is the Network Degraded Pod Count, which tracks the logical isolation boundaries of the cluster. This metric measure how many latency-sensitive pods are placed onto the network-degraded nodes.

$$S_{\text{degraded}} = \sum_{p \in G_{\text{checkout}}} f(p)$$

$$f(p) = \begin{cases} 1, & \text{if } N(p) \in D_{\text{tier}} \\ 0, & \text{otherwise} \end{cases}$$

Where:

- S_{degraded} is the total number of pod placed on degraded nodes, observed during a descheduling sweep.
- G_{checkout} represents the set of active pods with network-group label *checkout-flow* identifier.
- $f(p)$ is a binary placement evaluation function for pod p .
- $N(p)$ returns the worker node hosting pod p .
- D_{tier} represents the set of simulated network-degraded nodes (worker-4 and worker-5).

An un-aware baseline policy may produce an S_{degraded} value greater than zero if relocating a *checkout-flow* pod to a network degraded node (worker-4 and worker-5). In contrast, an ideal successful scenario of network-aware policy should maintain $S_{\text{degraded}} = 0$, demonstrating that latency-sensitive workloads remain confined to the high-performance node while only generic network-unaware workloads are allowed to migrate to the degraded node tier. Similar to latency, this metric is evaluated across all three scenarios to consistently track the logical isolation of the network-sensitive *checkout-flow* pods.

M5: Cluster Stranding Score (S) Because multi-dimensional defragmentation optimizes for the geometric "shape" of resource utilization rather than global absolute spread, Standard Deviation (σ_R) becomes an inaccurate measure of success. To properly evaluate the *ResourceDefragmentation* scenario, we reuse the Cluster Stranding Score (S), previously defined in Section 4.1.3.

As established, S aggregates the absolute difference between CPU and Memory utilization fractions across all active nodes:

$$S = \sum_{n \in N_{\text{active}}} |u_{n,\text{cpu}} - u_{n,\text{memory}}|$$

This metric completely replaces σ_R in the *ResourceDefragmentation* evaluation. Crucially, the score is summed rather than averaged. Averaging the imbalance across nodes can artificially mask successful consolidation (e.g., if three fragmented nodes are condensed into one node with the same local imbalance, the mean remains unchanged despite two nodes being successfully reclaimed). By using the sum, a lower Stranding Score accurately indicates a reduction in cluster-wide stranded capacity, rewarding geometric symmetry and node reclamation without being penalized for intentional variance in absolute global spread.

4.3.5 Experimental Results: LowNodeUtilization Scenario

The primary objective of the network-aware descheduler is to maintain optimal cluster resource utilization without severely degrading the communication latency of sensitive workloads. To validate this, we execute the *LowNodeUtilization* scenario across three distinct strategies: a baseline (no network awareness), topology-aware filtering, and the proposed latency-aware filtering.

Scenario	Evicted Pods	CPU StdDev (%)	MEM StdDev (%)	Avg Latency	Degraded Pods
Initial State	-	23.6	23.5	~4.0-20.0 ms	0
a1: Baseline	6	15.7	13.9	61.717 ms	1
a2: Topology L1	6	15.7	13.9	61.271 ms	1
a3: Topology L2	6	15.7	13.9	59.477 ms	1
a4: Topology L1+L2	6	15.7	13.9	67.305 ms	1
a5: Latency L1	6	15.4	14.3	4.195 ms	0
a6: Latency L2	6	15.4	14.3	3.017 ms	0
a7: Latency L1+L2	6	15.4	14.3	2.649 ms	0

Table 20. Descheduler Performance: LowNodeUtilization Scenario (MinBetterPercentage=75%)

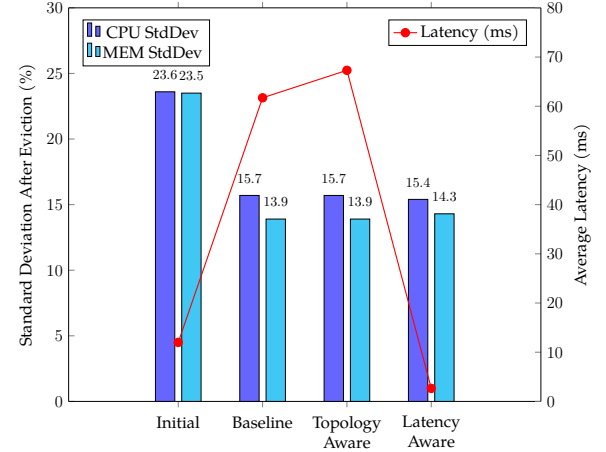


Figure 19. Cluster Balance vs. Network Performance.

Table 20 presents the raw performances of the evaluation using the optimal configuration (**MinBetterPercentage = 75%**).

As demonstrated in Table 20, all evaluated strategies successfully reduced the CPU and Memory Standard Deviation across the cluster. This indicates that the core balancing objective of the descheduler was achieved, regardless of the filtering strategy applied.

Resource Balance vs. Network Performance While all strategies achieved equivalent resource balancing, their impact on network performance varied drastically. Figure 19 illustrates this critical trade-off.

The baseline and topology-aware strategies successfully balanced the cluster but caused a catastrophic increase in average latency (spiking from ~4ms to

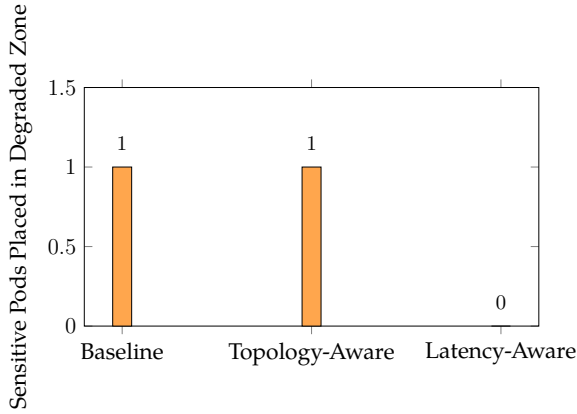


Figure 20. Number of sensitive pods placed into a network-degraded zone.

over 60ms). In contrast, the latency-aware strategies (a5-a7) achieved near equal cluster balance while maintaining the latency (~ 3 ms).

The massive latency spikes observed in the topology-aware tests are primarily caused by the topology model’s inability to detect actual network degradation. As visualized in Figure 20, the topology model only evaluates physical zone boundaries (e.g., Cross-Zone to Cross-Zone) and assumes all links are perfectly healthy, it considered degraded nodes as healthy one, resulting on lower or equal network cost calculation. The proposed latency-aware solved this issue by querying Prometheus for real-time metrics, correctly detecting the 160ms degradation and blocking the eviction.

Limitations of Pre-Eviction Filtering (L1)

During testing, we also discovered that the global Pre-Eviction filter (L1) is highly sensitive to configuration heuristics. Table 21 compares the performance of the L1 filter under different *MinBetterPercentage* thresholds against the strategy-embedded L2 filter.

When the L1 filter was configured with a loose threshold (50%), it evaluated all schedulable nodes in the cluster and

Configuration	Degraded Pods	Avg Latency (ms)
L1 Pre-Eviction (MinBetter = 50%)	1	63.696 ms
L1 Pre-Eviction (MinBetter = 75%)	0	4.195 ms
L2 Strategy Filter (No Tuning Req.)	0	3.017 ms

Table 21. Impact of MinBetterPercentage on L1 Pre-Eviction Filters

determined that 2 out of the 4 available candidate nodes offered equal or better latency. Because this satisfied the 50% minimum threshold requirement, the L1 filter allowed the eviction. However, when the pod was rescheduled, the *kube-scheduler* utilized its default *LeastAllocated* strategy. Since the degraded node in the cluster was also completely empty, the scheduler simply placed the pod there to balance CPU and memory resources, completely destroying the workload’s latency.

To mitigate this global-filtering flaw, the L1 threshold had to be strictly tuned to 75%. In contrast, the L2 filter embeds network-awareness directly into the destination-scoring logic. By evaluating only the exact target nodes rather than the entire cluster, L2 successfully filters out irrelevant "decoy" nodes and provides a much tighter safety net in this scenario, protecting the workload without requiring manual heuristic tuning.

No Network-Unaware Pods Scenario

The previous scenario included generic network-unaware pods, which provided the descheduler with a safe alternative to balance the node without touching the network sensitive workloads. To rigorously test the descheduler’s logic, we executed a variant where the overutilized nodes were populated exclusively only by network-sensitive pods.

Table 22 presents the evaluation results when no network-unaware pods were available to be sacrificed.

This scenario forces the descheduler to make a definitive choice between

Scenario	Evicted Pods	CPU StdDev (%)	MEM StdDev (%)	Avg Latency (ms)	Degraded Pods
Initial State	-	23.6	21.2	~4.0 ms	0
a1: Baseline	6	13.1	13.9	69.073 ms	2
a4: Topology L1+L2	6	13.1	13.9	61.249 ms	2
a7: Latency L1+L2	0	23.6	21.2	5.102 ms	0

Table 22. Descheduler Performance: The "Hard Tradeoff" Scenario (No Network-Unaware Pods)

cluster symmetry and network performance. As shown in Table 22, the Baseline and Topology-Aware strategies blindly prioritized CPU balancing. They evicted 6 critical pods, successfully balancing the cluster by dropping the CPU and Memory Standard Deviation to $\sim 13\%$. However, they placed 2 of those sensitive pods directly into the degraded zone, severely degrading the workload’s performance (spiking average latency to $\sim 50\text{ms}$).

In contrast, the Latency-Aware strategies evaluated the critical pods, detected that evicting any of them would subject them to degraded network conditions, and strictly blocked all evictions (0 Evicted Pods). By intentionally refusing to evict, the Latency-Aware descheduler chose to leave the cluster imbalanced (Std-Dev remained at $\sim 21\text{-}23\%$) to guarantee the network integrity of the application. This conclusively proves that the proposed implementation strictly prioritizes application latency over infrastructure symmetry.

4.3.6 Experimental Results: HighNodeUtilization Scenario

HighNodeUtilization scenario presents a unique challenge for the descheduler. Unlike *LowNodeUtilization*, which spreads pods across the cluster, *HighNodeUtilization* seeks to pack workloads by evicting pods from underutilized nodes and packing them onto overutilized ones. This relies heavily on the *kube-scheduler*’s *MostAllocated* scoring strategy, which creates a centered gravity to heavy load nodes when scheduling pod.

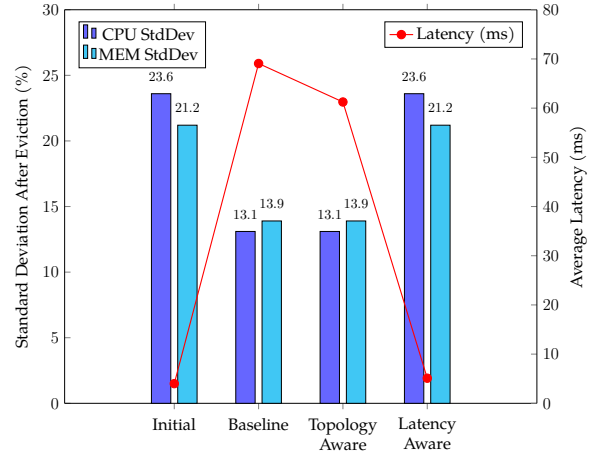


Figure 21. Cluster Balance vs. Network Performance (No Network-Unaware Pods Scenario).

Scenario	Evicted Pods	CPU StdDev (%)	MEM StdDev (%)	Active Nodes	Avg Latency	Degraded Pods
Initial State	-	13.4	11.2	5	~3.0 ms	0
b1: Baseline (No Net-Aware)	4	22.1	13.8	3	126.282 ms	2
b2: Topology L1	4	22.1	13.8	3	123.050 ms	2
b3: Topology L2	4	22.1	13.8	3	126.403 ms	2
b4: Topology L1+L2	4	22.1	13.8	3	129.800 ms	2
b5: Latency L1	2	17.7	12.5	5	2.728 ms	0
b6: Latency L2	2	17.7	12.5	5	2.676 ms	0
b7: Latency L1+L2	2	17.7	12.5	5	3.061 ms	0

Table 23. Descheduler Performance: HighNodeUtilization Scenario

Table 23 presents the raw performance of the descheduler when utilizing the strict (**MinBetterPercentage = 75%**).

As we observe in Table 23, the Baseline and Topology-Aware strategies successfully packed the cluster. By bin-packing the pods, they successfully emptied 2 machines (reducing active nodes from 5 to 3), which naturally caused the CPU and Memory Standard Deviations to increase. However, to achieve this node reclamation, they blindly placed 2 sensitive pods into the degraded zone, causing latency to spike to $\sim 130\text{ms}$. In contrast, the Latency-Aware strategies successfully blocked the dangerous evictions. While this meant the cluster could not be fully packed (leaving 5 active nodes), it perfectly preserved the $\sim 3\text{ms}$ latency.

MinBetterCandidatePercent Threshold Tuning for Stricter Evaluation Because the descheduler cannot guarantee final

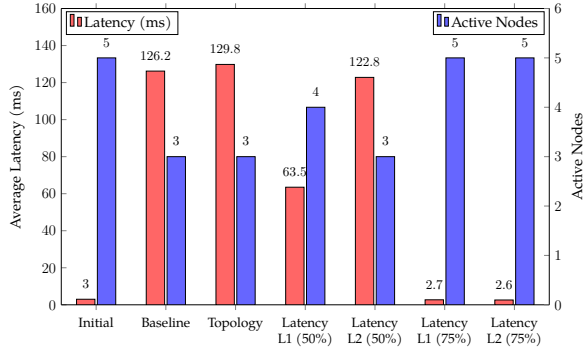


Figure 22. Performance comparison of *MinBetterCandidatePercent* Tuning

placement, it must assess the statistical risk of the *kube-scheduler*'s behavior. We best demonstrate the necessity of the strict 75% threshold by analyzing the catastrophic failure of the Latency-Aware L2 filter when configured with a looser 50% threshold.

As shown in Table 23, both the L1 and L2 latency filters failed when configured with a loose 50% threshold. When evaluating the L1 Pre-Eviction filter, it scanned 4 potential candidate nodes and found that 2 of them (worker-2 and worker-3) offered equal or better latency. Because this mathematically satisfied the 50% threshold, L1 authorized the evictions. However, the *kube-scheduler* subsequently placed one of the sensitive pods into the degraded zone, causing average latency to spike to ~63ms.

Same case happen to latency L2 filter at 50% that had ~122ms latency spike matching the spike in baseline and topology filter. This occurred due to an architectural boundary between the descheduler and the *kube-scheduler*. The L2 filter strictly evaluated the three highly-utilized target nodes and found that one of them (worker-3) was healthy (0ms latency). Because 1 out of 3 targets satisfied the loose 50% minimum threshold, the filter approved the eviction. However, once the pod was returned to

the scheduling queue, the *kube-scheduler*'s *MostAllocated* algorithm bypassed the healthy node and packed the pods onto the degraded nodes because they had a higher usage allocation.

The 75% threshold completely neutralized this threat for both filters, creating a need for minimum 3 and 2 better candidates node for L1 and L2 sequentially. When the L1 filter evaluated the entire cluster under the 75% configuration, it found that only 2 out of 4 of the schedulable nodes were safe, immediately blocking the eviction. Similarly, when the L2 filter re-evaluated its specific overutilized target nodes, it recognized that only 1 out of 3 of the available destinations were network-safe. Falling far short of the strict 75% safety requirement, both filters successfully blocked the network sensitive pods from being evicted.

However, this network protection came as a direct tradeoff for cluster bin-packing efficiency. As illustrated by the "Active Nodes" metric in Figure 22, the descheduler's refusal to evict the critical pods meant that the cluster could not be fully packed. While the Baseline strategy successfully emptied 2 machines to save infrastructure resources by reducing active nodes to 3, the Latency-Aware strategies were forced to leave all 5 nodes active to maintain the network latency. This tradeoff proves that our network aware descheduler explicitly prioritizing application health over infrastructure cost savings when a safe placement path cannot be mathematically guaranteed.

4.3.7 Experimental Results: ResourceDefragmentation Scenario

The *ResourceDefragmentation* scenario evaluates the descheduler's ability to correct multi-dimensional resource skew (CPU vs. Memory) without relying on strict utilization thresholds. This plugin uses the

Scenario	Evictions	Stranding Score	Avg Latency	Degraded Pods
Initial State	-	2.535	~8.0 ms	0
r1: Baseline	2	0.995	167.216 ms	2
r2: Topology L1	2	0.995	165.783 ms	2
r3: Latency L1	2	0.895	8.762 ms	0

Table 24. Descheduler Performance: ResourceDefragmentation Scenario

Resource Defragmentation algorithm to specifically target pods whose eviction will mathematically unlock trapped capacity on skewed nodes.

Because this algorithm optimizes for the geometric "shape" of resource utilization rather than global absolute spread, we evaluate its performance using the **Cluster Stranding Score (S)**. A lower Stranding Score indicates a better balanced, symmetrical cluster.

Prior to the descheduler execution, the cluster was forced into a highly fragmented state (Stranding Score: **2.535**), with nodes exhibiting massive orthogonal skews (e.g., 0.90 CPU vs 0.51 Mem, and 0.10 CPU vs 0.82 Mem).

Table 24 presents the raw performance of the descheduler across the three filtering strategies.

As shown in Table 24, the baseline *ResourceDefragmentation* plugin successfully identified the orthogonal CPU and Memory skews within the cluster and executed 2 evictions to aggressively balance the nodes (reducing the Stranding Score from 2.535 down to 0.995). However, because the Resource Defragmentation algorithm strictly optimizes for mathematical resource symmetry, it eagerly selected the network-sensitive *payment* pods for eviction, predicting that moving them to the memory-heavy degraded nodes would perfectly resolve the cluster's resource skew. Consequently, the baseline strategy placed 2 critical pods directly into the network-degraded zone, causing a catastrophic latency spike to ~167ms.

The Topology-Aware L1 filter failed

to prevent this. Because the topology model relies on static physical boundaries, it was entirely blind to the active network degradation. It approved the eviction of the critical pods, resulting in the exact same mathematical resource balance and the exact same network latency violation (~165ms) as the baseline.

Our proposed Latency-Aware L1 filter achieved a much better outcome: it successfully preserved the application's network latency while simultaneously allowing the descheduler to fix the cluster's resource fragmentation. When the Resource Defragmentation algorithm initially selected the sensitive *payment* pods for eviction, our Latency-Aware filter queried the real-time Prometheus metrics, detected the massive degradation on the target nodes, and strictly blocked the eviction.

Because the Kube-Descheduler framework operates iteratively, blocking the *payment* pods forced the Resource Defragmentation algorithm to evaluate the next-best candidates: the generic *network-unaware-cpu* pods. Because these network-unaware pods were not part of the sensitive network group, our filter safely allowed their eviction. By actively steering the descheduler away from the critical workloads, we forced the cluster to resolve its resource fragmentation using network-unaware pods. This resulted in an even better reduction in resource skew (Stranding Score dropped to 0.895) while completely protecting the network integrity of the application (0 degraded pods, ~8ms latency).

Defragmentation Performance and Network Latency Trade-offs Figure 23 visually summarizes the performance of the three filtering strategies, highlighting our Latency-Aware plugin's ability to decouple cluster resource balancing from network

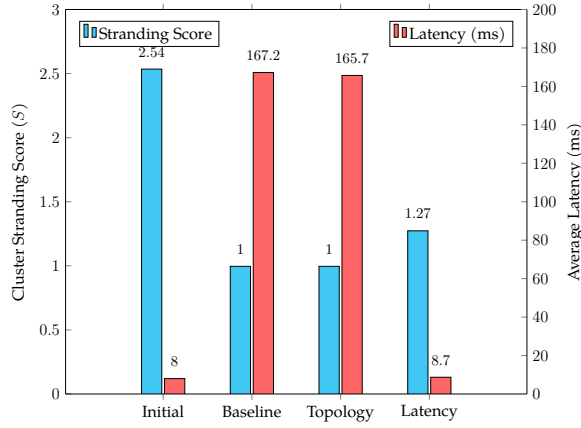


Figure 23. Cluster Stranding Score (S) vs. Network Performance.

performance degradation.

The results from the Resource Defragmentation scenario mathematically prove that our Latency-Aware plugin does not inherently compromise the core goal of the defragmentation algorithm. Instead of blindly moving critical workloads to satisfy a mathematical equation, the plugin intelligently evaluates network safety and blocks evictions that violate it. As demonstrated by the 0.895 final Stranding Score, the presence of these network-unaware pods allowed the steered approach to achieve a slightly superior geometric resource balance compared to the baseline. However, as established in our earlier Hard Tradeoff experiments, if no such network-unaware pods were available, the Latency-Aware plugin would actively choose to leave the cluster in a fragmented state (higher Stranding Score) to guarantee network integrity like showed in 22

4.4 End-to-End Combined Descheduler Use Case

This section presents an end-to-end use case that combines the three proposed mechanisms in a single Descheduler profile. The purpose is to clarify the operational position of each technique

in one descheduling pass. *ResourceDefragmentation* provides the request-space optimization objective at the *Balance* extension point, while *ActualUsageEvictor* and *NetworkCostEvictor* act as pre-eviction safeguards that prevent busy or network-sensitive Pods from becoming the final eviction.

4.4.1 Scenario Objective

The use case is not introduced as an additional quantitative benchmark. Instead, it connects the mechanisms evaluated in the previous sections into one realistic microservice placement. The cluster contains CPU-skewed and memory-skewed Nodes, so a memory-shaped Pod from a fragmented source Node can improve request-space packing when evicted and rebound by the Kubernetes scheduler. At the same time, not every resource-useful Pod is safe to evict. A foreground API Pod may be busy at runtime, and an idle Pod may still be risky to move if it is co-located with a low-latency dependency.

The combined profile therefore represents a constrained optimization flow. *ResourceDefragmentation* identifies resource-useful eviction opportunities, *ActualUsageEvictor* removes candidates whose live usage is high relative to their requests, and *NetworkCostEvictor* removes candidates whose possible rebinding would degrade dependency locality. The final eviction is the Pod that remains useful for defragmentation while satisfying both operational guards.

4.4.2 Initial Cluster and Application Placement

The scenario uses six worker Nodes grouped into two topology zones. Nodes *worker-1*, *worker-2*, and *worker-3* are labeled as *zone-a* and act as CPU-skewed expected landing targets. Nodes *worker-4* and *worker-5* are memory-skewed background

Table 25. End-to-End Use Case: Aggregate Initial Cluster Placement

Nodes	Zone	Resident Workloads	Placement Role
worker-1– worker-3	zone- a	Each Node hosts two CPU-shaped Pods, each requesting 750 <i>m</i> /25 <i>Mi</i>	CPU-skewed expected landing targets
worker-4– worker-5	zone- b	Each Node hosts two memory-shaped Pods, each requesting 50 <i>m</i> /240 <i>Mi</i>	Memory-skewed background Nodes
worker-6	zone- b	Hosts API, chatty, cache, and safe-memory Pods with total requests 200 <i>m</i> /760 <i>Mi</i>	Fragmented source Node

Table 26. End-to-End Use Case: Source Node Pods

Pod	CPU Re-request	Memory Request	Runtime State	Network Role
workload-api	50 <i>m</i>	300 <i>Mi</i>	Busy foreground API traffic	No network group
workload-chatty	50 <i>m</i>	240 <i>Mi</i>	Idle	checkout network group
workload-cache	50 <i>m</i>	40 <i>Mi</i>	Idle	checkout network group
workload-mem-safe	50 <i>m</i>	180 <i>Mi</i>	Idle	No network group

Nodes in *zone-b*. The fragmented source Node is *worker-6*, also in *zone-b*. The aggregate initial placement is summarized in Table 25.

This layout creates complementary resource shapes across the cluster. The first three Nodes have high requested CPU with little requested memory, so they are suitable expected targets for a memory-shaped Pod from the source Node. The next two Nodes keep the cluster heterogeneous and prevent the example from becoming a simple empty-node drain. The source Node hosts four application Pods from the combined-plugin workload, as summarized in Table 26.

This placement makes the roles

of the three mechanisms visible. From a resource-only perspective, *workload-api* is the strongest memory-shaped candidate because it has the largest memory request on the source Node. *workload-chatty* is also useful for resource defragmentation, but it belongs to the same checkout communication group as *workload-cache*. *workload-mem-safe* has a smaller memory request than the first two candidates, but it is idle and has no network dependency label.

4.4.3 End-to-End Eviction Decision Flow

The combined Descheduler profile executes the following decision flow.

1. **ResourceDefragmentation candidate targeting.** *ResourceDefragmentation* first identifies *worker-6* as a source Node because its request-space shape is memory-skewed and can be improved by relocating one memory-shaped Pod toward a CPU-skewed target. In a resource-only interpretation, *workload-api* offers the highest projected target bin-score improvement because its 300 *Mi* memory request gives the strongest shape correction. The next useful candidates are *workload-chatty* and *workload-mem-safe*, which still provide memory-shaped relocation opportunities.
2. **Runtime guard protects API.** A resource-useful candidate is not automatically evicted. It must satisfy the configured pre-eviction filter chain before becoming the final eviction. *ActualUsageEvictor* reads the latest Metrics API sample and compares each scoped Pod’s CPU and memory usage with its declared requests. Since *workload-api* is serving foreground API traffic and exceeds the configured 0.80 usage-to-request

threshold, the actual-usage guard prevents it from becoming the final eviction. The Descheduler therefore does not evict the busy API Pod.

3. **NetworkCostEvictor protects checkout locality.** After the busy API Pod is guarded, *workload-chatty* remains a useful memory-shaped candidate and passes the actual-usage guard because it is idle. However, it carries *network-group = checkout*. *NetworkCostEvictor* discovers *workload-cache* as a dependency Pod with the same group and computes the current communication cost from the candidate's current Node to the dependency placement. Because both Pods are initially co-located on *worker-6*, the current communication cost is zero. The filter then evaluates possible schedulable Nodes as rebinding candidates; moving *workload-chatty* away from *worker-6* would increase the cost, especially for cross-zone placements. Since too few candidate Nodes preserve or improve the current cost under the configured threshold, the network-aware guard prevents this candidate from becoming the final eviction.
4. ***workload-mem-safe* becomes the final eviction target.** The remaining safe candidate is *workload-mem-safe*. It is idle, has no *network-group* label, and still contributes a memory-shaped relocation opportunity for the defragmentation objective. It therefore satisfies both pre-eviction safeguards and becomes the final eviction target. The eviction returns the Pod to the Kubernetes scheduler; the Descheduler records the expected target for its own decision process, but the final binding is still performed by kube-scheduler.

4.4.4 Discussion

This end-to-end scenario shows that the proposed architecture does not replace the resource objective with safety checks. Instead, it constrains the resource objective with runtime and network-locality information. *ResourceDefragmentation* supplies the direction of optimization by identifying a fragmented source Node and selecting Pods whose request shape can improve the target bin score. *ActualUsageEvictor* adds runtime safety by preventing eviction of Pods that are currently busy relative to their declared requests. *NetworkCostEvictor* adds locality safety by rejecting Pods whose communication group would likely be rebound to a worse network placement.

The combined result is not necessarily the Pod that is most optimal under resource shape alone. In this scenario, the most resource-useful candidate is the busy API Pod, and the next candidate is network-sensitive. The final eviction is therefore *workload-mem-safe*: a Pod that still helps defragmentation, but does so under operational constraints. This illustrates the intended position of each proposed technique in the Descheduler pipeline. The strategy determines where resource correction is useful, the pre-eviction filters determine which candidates remain safe to evict, and kube-scheduler performs the final placement after the eviction.

5. CONCLUSION

This section summarizes the findings and proposes future research directions based on the evaluation results.

5.1 Conclusion

This research designed and evaluated a multi-level Kubernetes Descheduler architecture that combines request-based resource defragmentation, actual-usage pre-eviction protection, and network-

latency-aware eviction filtering.

Identifying fragmented Nodes.

ResourceDefragmentation identifies fragmented Nodes directly from their request-space resource shape, without a utilization threshold. It selects drain candidates from average utilization and CPU-memory bin quality, orders them by the Resource Imbalance Index (RII) and Free-Space Index (FSI), and then evicts the Pod whose predicted landing Node, found through *predictSchedulerTarget*, has the best projected bin score. A target ceiling and pack-upward guard prevent overfilling a Node or relocating a workload to an emptier one. This single predicted-target criterion is sufficient to drive effective packing and defragmentation.

Effectiveness against the baseline.

ResourceDefragmentation reclaimed four Nodes in S1, three in S2, and one in S4, all cases where *HighNodeUtilization* performed no eviction. It cut stranding by roughly 74% in S2 ($3.727 \rightarrow 0.971$) and 66% in S4 ($3.243 \rightarrow 1.112$), and by 91% in the heterogeneous-drain S5 ($1.338 \rightarrow 0.123$) while reclaiming the source Node with only two evictions against the baseline’s three. It also preserved or increased balanced headroom in every fragmentation scenario, every run converged to a zero-eviction pass within three passes, and no run left a Pod Pending.

Actual-usage filtering and application protection. Our evaluation demonstrates that runtime usage can be incorporated into the Kubernetes Descheduler as a pre-eviction safety gate without modifying the candidate ordering of the active balancing strategy. The *Actual Usage Evictor* blocked the busy API Pod under *ResourceDefragmentation*, *HighNodeUtilization*, and *LowNodeUtilization*, while idle candidates remained eligible. Over a common 30 s post-event window, the control failure rates ranged

from 16.67% to 17.24%, and each control experienced approximately 4.8 s of service disruption. Every treatment recorded 0% failure and no disruption. The placement cost varied by strategy. *ResourceDefragmentation* retained an eviction but lost the 0.816 stranding reduction produced by moving the API Pod. *HighNodeUtilization* reclaimed two rather than three source Nodes. *LowNodeUtilization* substituted an idle fallback and retained the same three evictions and 85.7% memory-deviation reduction. These results show that runtime protection can preserve application availability while allowing continued Descheduler progress when a compatible idle candidate remains available.

Network-aware filtering and latency trade-offs. Our evaluation demonstrates that network-aware filtering can be effectively integrated into the Kubernetes Descheduler via a dual-layer architecture. We developed a system combining a global pre-eviction filter (*NetworkCostEvictor*) with strategy-level assessments, using real-time telemetry or topology labels to intercept latency-degrading evictions. Our analysis indicates that this architecture preserves communication efficiency without compromising resource-balancing objectives, provided non-network-sensitive pods are available to sacrifice. Specifically, our results show that the system capped latency at $\sim 1.2\text{ms}$ (load-balancing) and $\sim 6.1\text{ms}$ (multi-dimensional skews) while preventing all cross-region pod spills, avoiding the $\sim 82\text{ms}$ to $\sim 98\text{ms}$ latency penalties observed in upstream baselines. However, our evaluation of the *HighNodeUtilization* scenario also revealed a critical systemic limitation: isolated descheduler intent can be subverted by the network-blind default Kubernetes scheduler, which frequently misrouted evicted pods to degraded

nodes based on raw compute availability. This confirms that high-precision network locality requires explicit synchronization with the primary cluster scheduler. Ultimately, our findings prove that network protection and geometric resource balance can be achieved together when alternative eviction candidates exist; otherwise, administrators must strictly prioritize between full cluster defragmentation and network locality preservation.

Overall, the results support a simple conclusion: selecting eviction candidates by the balance of their predicted landing Nodes is enough to drive useful Node reclamation and dimensional defragmentation when combined with request feasibility, bounded utilization, pack-upward behavior, actual-usage filtering, and network-aware safety checks.

6. FUTURE WORK

- **Broader Resource Dimensions:** Extend the framework to additional resource dimensions, including storage and GPU, so that target prediction and bin-quality scoring can handle workloads beyond CPU and memory.
- **Scheduler Coordination:** Establish and evaluate scheduler coordination mechanisms that expose the predicted target, or a compatible scoring signal, to the kube-scheduler, ensuring that predicted-target placements align with actual scheduling decisions and eliminating the misalignment that currently bounds maximum defragmentation precision.
- **Pending-Pod Awareness:** Introduce Pending-pod awareness into the eviction pipeline so that drain candidates are selected with knowledge of

already-unschedulable workloads, ensuring evictions free the right resource type on the right Node to unblock Pending Pods rather than delaying them further.

- **Threshold Sensitivity Analysis:** Conduct a sensitivity analysis of the drain threshold, target ceiling, and related parameters across various workloads and cluster types to characterize their effect on node reclamation, stranding reduction, and convergence stability.
- **Broader Generalizability Evaluation:** Evaluate the plugin across broader cluster shapes, node heterogeneity, and workload mixes to assess generalizability beyond the five scenarios studied.
- **Traffic-Volume Weighting:** Weight network costs using service-mesh traffic volumes so that heavily used dependency paths receive stronger protection.

A. RESOURCE DEFRAGMENTATION BALANCE FILTER ALGORITHM

```

1 Function: ResourceDefragmentationBalance
2 Input:
3   nodes
4   maxEvictions
5   drainThreshold
6   targetUtilization
7 Output:
8   evictedPods
9
10  states <- BuildRequestBasedNodeStates(nodes)
11  drainCandidates <- empty list
12
13  for each worker node n in nodes do
14    avg <- AverageUtilization(states[n])
15    bin <- BinScore(states[n])
16    rii <- ResourceImbalanceIndex(states[n])
17    fsi <- FreeSpaceIndex(states[n])
18    pods <- GetEvictablePods(n)
19
20    if pods is not empty and
21       (avg < drainThreshold or
22        bin < drainThreshold) then
23      priority <- 0.5 * abs(rii) +
24                  0.5 * (1 / (fsi +
25                               ↪ epsilon))

```

```

25     Add(drainCandidates, {n, pods,
    ↪ priority})
26     end if
27   end for
28
29   SortDescending(drainCandidates, by priority)
30   bins <- empty set
31   evictionCount <- 0
32
33   for each drain in drainCandidates do
34     if evictionCount >= maxEvictions then
    ↪ break
35     if drain.node in bins then continue
36
37     remaining <- drain.pods
38     while remaining is not empty and
39       evictionCount < maxEvictions do
40       bestPod <- empty
41       bestTarget <- empty
42       bestScore <- negative infinity
43
44       for each pod p in remaining do
45         target <- PredictFeasibleTarget(
46           p,
47           drain.node,
48           states,
49           targetUtilization,
50           requirePackUpward = true)
51
52         if target exists then
53           projected <-
    ↪ StateAfterPlacement(states[target], p)
54           score <- BinScore(projected)
55           if score > bestScore then
56             bestPod <- p
57             bestTarget <- target
58             bestScore <- score
59           end if
60         end if
61       end for
62
63       if bestPod is empty then break
64
65       if Evict(bestPod) succeeds then
66         evictionCount <- evictionCount +
    ↪ 1
67         UpdateSourceAndTargetStates(
68           states, drain.node,
    ↪ bestTarget, bestPod)
69         Add(bins, bestTarget)
70       end if
71       Remove(remaining, bestPod)
72     end while
73   end for
74
75   return evictedPods

```

B. ACTUAL USAGE PRE-EVICTION FILTER ALGORITHM

```

1  Function: ActualUsagePreEvictionFilter
2  -----
3  Input:
4      P                // candidate Pod
5      config           // {cpuThreshold,
    ↪ memoryThreshold,
6                          missingRequestPolicy,

```

```

7                          namespaces,
    ↪ labelSelector}
8
9  Output:
10     Allow or Block
11
12  if P is outside namespace scope then
13     return Allow
14  end if
15
16  if P does not match labelSelector then
17     return Allow
18  end if
19
20  metrics (*$<leftarrow$*) ReadPodMetrics(P)
21  if metrics is unavailable then
22     return Block
23  end if
24
25  requestCPU (*$<leftarrow$*) 0
26  requestMemory (*$<leftarrow$*) 0
27  usageCPU (*$<leftarrow$*) 0
28  usageMemory (*$<leftarrow$*) 0
29  cpuRequestMissing (*$<leftarrow$*) false
30  memoryRequestMissing (*$<leftarrow$*) false
31
32  for each regular container c (*$in$*)
    ↪ P.containers do
33     if metrics for c are incomplete then
34        return Block
35     end if
36
37     usageCPU (*$<leftarrow$*) usageCPU +
    ↪ metrics[c].cpu
38     usageMemory (*$<leftarrow$*) usageMemory +
    ↪ metrics[c].memory
39
40     if c.requestCPU is missing then
41        cpuRequestMissing (*$<leftarrow$*) true
42     else
43        requestCPU (*$<leftarrow$*) requestCPU +
    ↪ c.requestCPU
44     end if
45
46     if c.requestMemory is missing then
47        memoryRequestMissing (*$<leftarrow$*)
    ↪ true
48     else
49        requestMemory (*$<leftarrow$*)
    ↪ requestMemory + c.requestMemory
50     end if
51  end for
52
53  cpuBusy (*$<leftarrow$*) EvaluateDimension(
54     usageCPU,
55     requestCPU,
56     cpuRequestMissing,
57     config.cpuThreshold,
58     config.missingRequestPolicy)
59
60  memoryBusy (*$<leftarrow$*) EvaluateDimension(
61     usageMemory,
62     requestMemory,
63     memoryRequestMissing,
64     config.memoryThreshold,
65     config.missingRequestPolicy)
66
67  if cpuBusy or memoryBusy then
68     return Block
69  end if

```



```

70
71 return Allow
72
73 Function: EvaluateDimension
74 -----
75 Input:
76     usage, request, requestMissing, threshold,
    ↪ policy
77
78 Output:
79     busy
80
81 if requestMissing then
82     return policy == Block
83 end if
84
85 return usage / request (*$geq$*) threshold

```

C. NETWORK-AWARE PRE-EVICTION FILTER ALGORITHM

```

1 Function: NetworkAwareShouldAllowEviction
2 -----
3 Input:
4     N                // set of all cluster nodes
5     config           /* descheduler policy
    ↪ configuration:
6         { thresholds,
    ↪ targetThresholds,
7             labelKey, minBetterPct,
    ↪ excludeSameOwner,
8             costProvider,
9             L1Enabled, L2Enabled } */
10
11 Output:
12     None
13
14 // Phase 1: Initialize Cost Provider
15 if latencyMetrics is configured then
16     matrix (*$leftarrow$*)
    ↪ QueryPrometheus(query, sourceLabel,
    ↪ targetLabel)
17     costProvider (*$leftarrow$*)
    ↪ LatencyCostProvider(matrix)
18 else
19     costProvider (*$leftarrow$*)
    ↪ TopologyCostProvider()
20 end if
21 nodesMap (*$leftarrow$*) {n.name
    ↪ (*$rightarrow$*) n | n (*$in$*) N}
22
23
24 // Phase 2: Classify Nodes by Utilization (only
    ↪ apply if L2 enabled)
25 overutilized (*$leftarrow$*) (*$emptyset$*)
26 underutilized (*$leftarrow$*) (*$emptyset$*)
27 for each node n (*$in$*) N do
28     usage(n) (*$leftarrow$*) (*$Sigma$*)
    ↪ requests of all pods on n
29     util(n) (*$leftarrow$*) usage(n) /
    ↪ allocatable(n) * 100
30     if util(n) > targetThresholds then
31         overutilized (*$leftarrow$*)
    ↪ overutilized (*$cup$*) {n} // for
    ↪ highnodeutilization strategy
32     else if util(n) < thresholds then

```

```

33         underutilized (*$leftarrow$*)
    ↪ underutilized (*$cup$*) {n}. // for
    ↪ lownodeutilization strategy
34     end if
35 end for
36
37 if nodeutilizationStrategy = highnodeutilization
    ↪ then
38     destNodes (*$leftarrow$*) overutilized
39 else if nodeutilizationStrategy =
    ↪ lownodeutilization then
40     destNodes (*$leftarrow$*) underutilized
41 else
42     destNodes (*$leftarrow$*) N
43 end if
44
45 // Phase 3: Select and Filter Candidate Pods
46 for each node s (*$in$*) sourceNodes do
47     candidates (*$leftarrow$*)
    ↪ GetEvictablePods(s)
48     SortByPriority(candidates) //
    ↪ lowest priority first
49
50     for each pod P (*$in$*) candidates do
51         if L2Enabled then
52             D (*$leftarrow$*)
    ↪ FindDependencyPods(P, labelKey, N,
    ↪ excludeSameOwner)
53             if D (*$neq$*) (*$emptyset$*) then
54                 currentCost (*$leftarrow$*)
    ↪ ComputePlacementCost(
55                     nodesMap[P.nodeName], D,
56                     nodesMap, costProvider
57                 )
58                 betterCount (*$leftarrow$*) 0
59                 for each node c (*$in$*)
60                     candCost (*$leftarrow$*)
    ↪ ComputePlacementCost(c, D, nodesMap,
    ↪ costProvider)
61                     if candCost (*$leq$*)
    ↪ currentCost then
62                         betterCount
63                         ↪ (*$leftarrow$*) betterCount + 1
64                         end if
65                         end for
66                         minReq (*$leftarrow$*) max(1,
    ↪ (*$lfloor$*)|destNodes| * minBetterPct /
    ↪ 100(*$rfloor$*))
67                         if betterCount < minReq then
68                             continue //
    ↪ BLOCK: no good dest
69                             end if
70                         end if
71                     end if
72
73                     if L1Enabled then
74                         D (*$leftarrow$*)
    ↪ FindDependencyPods(P, labelKey, N,
    ↪ excludeSameOwner)
75                         if D (*$neq$*) (*$emptyset$*) then
76                             currentCost (*$leftarrow$*)
    ↪ ComputePlacementCost(
77                                 nodesMap[P.nodeName], D,
78                                 nodesMap, costProvider
79                             )
80
81                             betterCount (*$leftarrow$*) 0
82                             for each node c (*$in$*) N do

```

```

83         candCost (*$\leftarrow$*)
    ↪ ComputePlacementCost(c, D, nodesMap,
    ↪ costProvider)
84         if candCost (*$\leq$*)
    ↪ currentCost then
85             betterCount
    ↪ (*$\leftarrow$*) betterCount + 1
86             end if
87         end for
88         minReq (*$\leftarrow$*) max(1,
    ↪ (*$\lfloor$floor$*)|N| * minBetterPct /
    ↪ 100(*$\rfloor$*))
89         if betterCount < minReq then
90             continue
    ↪ // BLOCK: global filter
91         end if
92     end if
93 end if
94
95     Evict(P)
96
97 end for
98 end for

```

REFERENCES

Bagaa, M. et al. (2024). A multi-objective framework for power-aware scheduling in kubernetes. *Depot-e UQTR*.

Chen, M., Islam, M. T., Read, M. R., and Buyya, R. (2024). Trade: Network and traffic-aware adaptive scheduling for microservices under dynamics. *arXiv preprint arXiv:2411.05323*.

Guo, Y., Wei, L., Fan, C., Ma, Y., Zhao, X., and He, H. (2025). Joint optimization of container resource defragmentation and task scheduling in queueing cloud computing: A drl-based approach. *Future Internet*, 17(11):483.

Kubernetes Authors (n.d.a). *Limit Ranges*. <https://kubernetes.io/docs/concepts/policy/limit-range/>.

Kubernetes Authors (n.d.b). *Node-Pressure Eviction*. <https://kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/>.

Kubernetes Authors (n.d.c). *Pod Quality of Service Classes*. <https://kubernetes.io/docs/concepts/scheduling-eviction/pod-qos/>.

[//kubernetes.io/docs/concepts/workloads/pods/pod-qos/](https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/).

Kubernetes Authors (n.d.d). *Resource Bin Packing*. <https://kubernetes.io/docs/concepts/scheduling-eviction/resource-bin-packing/>.

Kubernetes Authors (n.d.e). *Resource Management for Pods and Containers*. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>.

Kubernetes SIG Scheduling (n.d.). *Descheduler for Kubernetes*. <https://github.com/kubernetes-sigs/descheduler>.

Larsson, O., Klein, C., and Elmroth, E. (2023). The impact of directed pod eviction on kubernetes resource utilization. In *2023 IEEE 17th International Conference on Service-Oriented System Engineering (SOSE)*, pages 78–87.

Marchese, A. and Tomarchio, O. (2022a). Communication aware scheduling of microservices-based applications on kubernetes clusters. In *Proceedings of the 12th International Conference on Cloud Computing and Services Science (CLOSER 2022)*, pages 190–198.

Marchese, A. and Tomarchio, O. (2022b). Network-aware container placement in cloud-edge kubernetes clusters. In *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 859–865. IEEE.

Marchese, A. and Tomarchio, O. (2024). Load-aware container orchestration on kubernetes clusters. In *Proceedings of the 14th International Conference on Cloud Computing and Services Science*, pages 92–102.

- Pusztai, T., Rossi, F., and Dustdar, S. (2021). Pogonip: Scheduling asynchronous applications on the edge. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*, pages 582–591.
- Xiao, Z., Song, W., and Chen, Q. (2013). Dynamic resource allocation using virtual machines for cloud computing environment. *IEEE Transactions on Parallel and Distributed Systems*, 24(6):1107–1117.